

Pandas

- Pandas is an open-source, high-performance, and easy-to-use Python library for data manipulation and analysis.
- The name Pandas comes from "Panel Data", a term used in econometrics for datasets with multiple dimensions.
- It is built on top of NumPy, which means it uses fast numerical operations under the hood but provides more powerful, user-friendly structures for working with labeled and relational data.

Core Data Structures in Pandas

Pandas has two primary data structures:

a) Series

- A one-dimensional labeled array.
- Similar to an Excel column or a Python list with indexes.
- Can hold any data type (integers, strings, floats, Python objects).

Example

```
import pandas as pd
s = pd.Series([10, 20, 30], index=["A", "B", "C"])
print(s)
```

b) DataFrame

- A two-dimensional labeled data structure with rows and columns.
- Similar to an Excel spreadsheet or a SQL table.
- Each column is essentially a Series object.

Example

```
data = {"Name": ["Anjali", "Nikhil", "Prashant"],
        "Age": [25, 28, 30],
        "City": ["Agra", "Delhi", "Jaipur"]}
df = pd.DataFrame(data)
print(df)
```

Why Pandas is important in Data Analysis?

a) Data Import & Export: Pandas can read data from:

- CSV, Excel, JSON, HTML
- SQL databases
- APIs
- Pickle files, Parquet, etc.

Example

```
df = pd.read_csv("data.csv") # Read CSV file
df.to_excel("output.xlsx")   # Save to Excel
```

b) Data Cleaning: In real-world data analysis, data is messy. Pandas makes it easy to:

- Handle missing values (`fillna()`, `dropna()`).
- Remove duplicates (`drop_duplicates()`).
- Convert data types (`astype()`).
- Rename columns and indexes.

Example

```
df['Age'].fillna(df['Age'].mean(), inplace=True)
```

c) Data Selection & Filtering: Pandas provides multiple ways to select and filter data:

- **Label-based indexing** with `.loc[]`
- **Position-based indexing** with `.iloc[]`
- Boolean conditions for filtering

Example

```
# Select rows where Age > 25
df_filtered = df[df['Age'] > 25]
```

d) Data Transformation

- Add new columns based on calculations.
- Apply functions to rows or columns with `.apply()`.
- Reshape data with `.pivot_table()` and `.melt()`.

Example

```
df['Age_in_5_years'] = df['Age'] + 5
```

e) Grouping & Aggregation: With `groupby()`, you can:

- Summarize data.
- Calculate statistics like sum, mean, min, max.

Example

```
df.groupby("City")["Age"].mean()
```

f) Time Series Analysis: Pandas has strong date and time manipulation capabilities, making it great for:

- Financial data analysis.
- Trend tracking over time.
- Resampling and shifting data.

Example

```
df['Date'] = pd.to_datetime(df['Date'])
df.set_index('Date', inplace=True)
df.resample('M').mean()
```

g) Speed & Integration

- **Performance:** Optimized with C extensions for fast data operations.
- **Integration:** Works seamlessly with NumPy, Matplotlib, Scikit-learn, and Seaborn.
- This makes Pandas the **central hub** of Python's Data Science ecosystem.

Why Every Data Analyst Needs Pandas

- **Without Pandas**
 - You'd spend hours writing loops to clean and process data.
 - Working with CSVs or SQL data would be tedious.
 - Data transformations and aggregations would require much more code.
- **With Pandas**
 - Few lines of code can do complex operations.
 - You can focus on analysis, not low-level coding.
 - You can handle small datasets and millions of rows with ease.

Important Points: Pandas is important because it:

- Simplifies complex data operations.
- Saves time in cleaning and preparing data.
- Integrates well with other data science tools.
- Handles real-world messy datasets efficiently.
- Empowers analysts to get insights faster.

Installing Pandas

- Pandas is a third-party Python library, so it's not included in Python by default.
- You need to install it using pip (Python's package manager) or conda (Anaconda's package manager).

a) Installing with pip (Recommended if you use standard Python): Make sure you have **Python** and **pip** installed.

You can check by running in your terminal/command prompt:

```
python --version
pip --version
```

Install Pandas:

```
pip install pandas
```

Upgrade Pandas (if already installed):

```
pip install --upgrade pandas
```

b) Installing with conda (Recommended if you use Anaconda): If you are using **Anaconda** or **Miniconda**, run:

```
conda install pandas
```

To update Pandas in Anaconda:

```
conda update pandas
```

c) Installing Along with NumPy: Since Pandas is built on top of NumPy, it's best to make sure NumPy is installed too:

```
pip install pandas numpy
```

or

```
conda install pandas numpy
```

d) Verifying the Installation: After installation, open Python (or Jupyter Notebook) and run:

```
import pandas as pd  
print(pd.__version__)
```

If it prints a version number without an error, Pandas is installed successfully.

Why Pandas is already installed in Anaconda?

- Anaconda comes with the Anaconda Distribution, which includes Pandas, NumPy, Matplotlib, Scikit-learn, Jupyter Notebook, and many other data science libraries.
- This means you can immediately run:

```
import pandas as pd  
print(pd.__version__)
```

If this works without an error and prints a version number, Pandas is installed.

NumPy vs Pandas

Feature	NumPy	Pandas
Purpose	Numerical computing (fast array operations)	Data manipulation & analysis (tabular/structured data)
Data Type	Works mainly with homogeneous numeric data	Works with heterogeneous data (numbers, strings, dates)
Data Structure	ndarray (N-dimensional array)	Series (1D), DataFrame (2D)
Indexing	Integer-based only	Label-based & integer-based indexing
Missing Data	Not handled natively (needs np.nan)	Built-in handling with NaN support
Data Sources	Mostly raw arrays/matrices	Can read from CSV, Excel, SQL, JSON, etc.

Use NumPy when:

- You need fast numerical computations (matrix operations, vectorized math).
- Your data is purely numeric and structured in arrays.
- You are working on linear algebra, FFT, image processing, or machine learning preprocessing.
- You need multi-dimensional arrays and high-performance math functions.

Example

```
import numpy as np
arr = np.array([1, 2, 3])
print(arr * 2)                # Fast element-wise multiplication
```

Use Pandas when:

- Your data is tabular (rows and columns) and may include mixed types (numbers, strings, dates).
- You need data cleaning, filtering, grouping, merging, or reshaping.
- You are working with real-world datasets from CSV, Excel, SQL, etc.
- You need label-based indexing and easy handling of missing values.

Example

```
import pandas as pd
df = pd.DataFrame({'Name': ['Amit', 'Priya'], 'Age': [25, 28]})
print(df[df['Age'] > 25])
```

Note:

- **NumPy** : Best for low-level, high-speed numerical computations.
- **Pandas** : Best for high-level data manipulation & analysis.

Pandas Series

- A Series in Pandas is a one-dimensional labeled array capable of holding any data type — integers, floats, strings, Python objects, etc.
- It has two main components:
 - Data (values you store)
 - Index (labels for each value — defaults to 0, 1, 2, ... if not given)

Creating a Series from a List

A Python list is an ordered collection of items. Pandas allows you to convert it into a Series easily.

Syntax

```
import pandas as pd
pd.Series(data, index=None, dtype=None, name=None)
```

where,

- **data** : Python list (or other data type)
- **index** : Custom labels (optional)
- **dtype** : Data type of values (optional)
- **name** : Name of the Series (optional)

Example: Basic Series from a List

```
import pandas as pd

fruits = ["Apple", "Banana", "Mango", "Orange"]
series1 = pd.Series(fruits)
print(series1)
```

Default index **0, 1, 2, 3** is created automatically.

Data type is object because we have strings.

Example: Series with Numeric List

```
marks = [85, 90, 78, 92]
series2 = pd.Series(marks)
print(series2)
```

Values are integers, so dtype is int64

Example: Series with Custom Index

```
cities = ["Agra", "Delhi", "Mumbai", "Kolkata"]
index_labels = ["City1", "City2", "City3", "City4"]
series3 = pd.Series(cities, index=index_labels)
print(series3)
```

Example: Series with Custom Name and Data Type

```
temperatures = [36.5, 37.0, 35.8, 36.2]
series4 = pd.Series(temperatures, name="Body Temperature", dtype="float32")
print(series4)
```

We gave a name to the Series and set a specific data type (float32).

Example: Series with Mixed Data Types

```
mixed_list = ["Learn2Earn Labs", 2025, 99.9, True]
series5 = pd.Series(mixed_list)
print(series5)
```

Mixed types are converted to object type for compatibility.

Notes:

- If no index is provided, Pandas creates a default integer index starting from 0.
- Lists can have any type of elements (strings, numbers, booleans, etc.).
- You can customize index, dtype, and name.

Creating a Series from NumPy Arrays

- You might already have data in a NumPy array from mathematical or statistical operations.
- Converting it to a Pandas Series allows you to:
 - Add meaningful labels (index)
 - Use Pandas' powerful methods (filtering, aggregation, plotting, etc.)
 - Integrate easily with DataFrames

Example: Basic Series from NumPy Array

```
import pandas as pd
import numpy as np

arr = np.array([10, 20, 30, 40])
series1 = pd.Series(arr)
print(series1)
```

Example: With custom index

```
arr = np.array([100, 200, 300])
series2 = pd.Series(arr, index=['A', 'B', 'C'])
print(series2)
```

Labels replace default numerical index.

Example: Float Array with dtype Change

```
arr = np.array([12.5, 15.7, 18.3])
series3 = pd.Series(arr, dtype='float32')
print(series3)
```

Forced dtype from float64 to float32

Example: Mixed Data (Converted to object type)

```
arr = np.array([1, 'Learn2Earn Labs', 99.5], dtype=object)
series4 = pd.Series(arr)
print(series4)
```

Mixed types → Pandas stores as object.

Example: NumPy Dates to Series

```
dates = np.array(['2025-01-01', '2025-01-02', '2025-01-03'], dtype='datetime64[D]')
series5 = pd.Series(dates)
print(series5)
```

NumPy date arrays are preserved as Pandas datetime.

Example: Student Marks

```
students = np.array([85, 90, 78, 92, 88])
names = ["Aman", "Rohit", "Priya", "Neha", "Kunal"]
marks_series = pd.Series(students, index=names, name="Final Marks")
print(marks_series)
```

Easily filter top scorers or calculate averages.

Example: Product Prices

```
products = np.array([55000, 32000, 25000, 8000])
product_names = ["Laptop", "Tablet", "Phone", "Smartwatch"]
price_series = pd.Series(products, index=product_names, name="Price in INR",
dtype="float32")
print(price_series)
```

Great for price comparison and discount calculations.

Example: City Temperature Readings (Can be used for seasonal trend analysis)

```
temperatures = np.array([32.5, 34.2, 30.8, 31.7])
cities = ["Agra", "Delhi", "Mumbai", "Kolkata"]
temp_series = pd.Series(temperatures, index=cities, name="Avg Temperature (°C)")
print(temp_series)
```


Example: Product Stock Data

```
stock_units = np.array([25, 40, 100, 60])
products = ["Laptop", "Tablet", "Phone", "Smartwatch"]
stock_series = pd.Series(stock_units, index=products, name="Available Units")
print(stock_series)
```

Helps identify which products need restocking.

Example: Timestamps (Daily Visitors)

```
dates = np.array(["2025-08-10", "2025-08-11", "2025-08-12", "2025-08-13"],
dtype="datetime64[D]")
visitors = np.array([1500, 1850, 1750, 2100])
visitors_series = pd.Series(visitors, index=dates, name="Daily Visitors")
print(visitors_series)
```

Perfect for time series analysis and trend plotting.

Creating a Series from a Dictionary

- A dictionary in Python stores key-value pairs.
- In a Pandas Series:
 - Keys become the index.
 - Values become the data.
- This is especially useful when you already have labeled data (like product prices, student marks, etc.).

Example: Basic Series from Dictionary

```
import pandas as pd
marks_dict = {"Aman": 85, "Rohit": 90, "Priya": 78}
series1 = pd.Series(marks_dict)
print(series1)
```

Keys become index, values become data.

Example: Custom Index (Reorder & Select Keys)

```
marks_dict = {"Aman": 85, "Rohit": 90, "Priya": 78}
series2 = pd.Series(marks_dict, index=["Priya", "Aman"])
print(series2)
```

Example: Missing Keys in Custom Index (NaN appears for keys not found in dictionary)

```
marks_dict = {"Aman": 85, "Rohit": 90}
series3 = pd.Series(marks_dict, index=["Aman", "Rohit", "Priya"])
print(series3)
```

Example: With Name and Data Type

```
product_prices = {"Laptop": 55000, "Tablet": 32000, "Phone": 25000}

series4 = pd.Series(product_prices, name="Price in INR", dtype="float32")
print(series4)
```

Added a name and forced all values to float.

Example: Mixed Data Types

```
student_info = {"Name": "Ritika", "Age": 22, "CGPA": 8.9}
series5 = pd.Series(student_info)
print(series5)
```

Mixed data stored as object type.

Example: Monthly Sales Data

```
sales_data = {
    "January": 150000,
    "February": 175000,
    "March": 160000,
    "April": 185000
}

sales_series = pd.Series(sales_data, name="Monthly Sales (INR)")
print(sales_series)
```

Useful for plotting monthly trends.

Example: City Temperature Readings

```
temperature_data = {
    "Agra": 32.5,
    "Delhi": 34.2,
    "Mumbai": 30.8,
    "Kolkata": 31.7
}

temp_series = pd.Series(temperature_data, name="Avg Temperature (°C)")
print(temp_series)
```

Helps compare temperatures across cities.

Example: Student Exam Scores

```
exam_scores = {  
    "Aman": 88,  
    "Rohit": 92,  
    "Priya": 79,  
    "Neha": 85  
}  
scores_series = pd.Series(exam_scores, name="Final Exam Marks", dtype="float32")  
print(scores_series)
```

Setting dtype=float32 allows decimal marks in future updates.

Example: Product Stock Levels

```
stock_levels = {  
    "Laptop": 25,  
    "Tablet": 40,  
    "Phone": 100,  
    "Smartwatch": 60  
}  
stock_series = pd.Series(stock_levels, name="Available Units")  
print(stock_series)
```

Helps in stock monitoring and restocking decisions.

Example: Daily Website Visitors

```
visitors_data = {  
    "2025-08-10": 1500,  
    "2025-08-11": 1850,  
    "2025-08-12": 1750,  
    "2025-08-13": 2100  
}  
visitors_series = pd.Series(visitors_data, name="Daily Visitors")  
print(visitors_series)
```

Can be directly used for trend analysis in web analytics.

Creating a Series from Scalars

- A scalar is a single value (e.g., a number, string, or boolean).
- When you create a Series from a scalar, Pandas repeats that value for each index label you provide.
- You must provide an index, otherwise Pandas won't know how many times to repeat the value.

Example: Repeating a Number

```
scalar_value = 100
index_labels = ["Aman", "Rohit", "Priya", "Neha"]
series1 = pd.Series(scalar_value, index=index_labels)
print(series1)
```

Same value is repeated for all index labels.

Example: Repeating a String

```
scalar_value = "Learn2Earn Labs"
index_labels = ["Course1", "Course2", "Course3"]

series2 = pd.Series(scalar_value, index=index_labels)
print(series2)
```

Useful when assigning a constant category to multiple records.

Example: With a Float and Custom Name

```
scalar_value = 36.5
index_labels = ["Day1", "Day2", "Day3", "Day4"]

series3 = pd.Series(scalar_value, index=index_labels, name="Body Temp (°C)",
dtype="float32")
print(series3)
```

Data type is explicitly set to float32.

Example: Boolean Values

```
scalar_value = True
index_labels = ["Task1", "Task2", "Task3"]

series4 = pd.Series(scalar_value, index=index_labels)
print(series4)
```

Could be used for marking a batch of tasks as completed.

Example: Using Dates

```
scalar_value = np.datetime64("2025-08-14")
index_labels = ["Event1", "Event2", "Event3"]

series5 = pd.Series(scalar_value, index=index_labels, name="Event Date")
print(series5)
```

Useful for assigning the same date to multiple events.

Indexing and Slicing in a Pandas Series

- A Series is a one-dimensional labeled array in pandas. Each element has two parts:
 - **Index (label / position)**: The "address" of the element. It can be numbers, strings, dates, etc.
 - **Value**: The data stored at that index.
- **Indexing** means selecting elements from a Series using either:
 - **Position-based indexing** (like Python lists).
 - **Label-based indexing** (using explicit index labels).
 - **Boolean indexing** (selecting elements that satisfy a condition).
 - **Fancy indexing** (selecting multiple items with an array/list of positions or labels).
- **Slicing** means extracting a range (subset) of elements from a Series. It can also be done by:
 - Position (using integers).
 - Labels (using index names).
 - Both `.iloc[]` and `.loc[]` methods provide precise slicing options.

Indexing in Series

```
import pandas as pd
```

```
s = pd.Series([100, 200, 300, 400, 500],
              index=["a", "b", "c", "d", "e"],
              name="Marks")
print(s)
```

Example: Indexing by position

```
print(s[0])
```

First element

Example: Indexing by label

```
print(s["c"])
```

Using index label

Example: Indexing multiple elements by position

```
print(s[[0, 2, 4]])
```

1st, 3rd, and 5th elements

Example: Indexing multiple elements by labels

```
print(s[["b", "e"]])
```

Example: Negative indexing

```
print(s[-1])
```

Last element

Example: Indexing with `.iloc[]` (position-based, safe)

```
print(s.iloc[2])
```

3rd element

Example: Indexing with `.loc[]` (label-based, safe)

```
print(s.loc["d"])
```

Access by label

Example: Boolean indexing (single condition)

```
print(s[s > 250])
```

Select values > 250

Example: Boolean indexing (multiple conditions)

```
print(s[(s >= 200) & (s <= 400)])
```

Example: Fancy indexing with a list of labels

```
print(s.loc[["e", "a", "c"]])
```

Arbitrary order

Slicing in Series – Point to note

- Python slicing [start:stop] → stop is **exclusive**.
- Pandas slicing by **labels** → stop is **inclusive**.
- .iloc[] (position-based) → behaves like Python.
- .loc[] (label-based) → includes both ends.

Example: Slice by position range

```
print(s[1:4])
```

2nd to 4th elements

Example: Slice by label range

```
print(s["b":"d"])
```

From b to d (inclusive)

Example: Slice with .iloc[] (position-based, exclusive end)

```
print(s.iloc[0:3])
```

First 3 elements

Example: Slice with .loc[] (label-based, inclusive end)

```
print(s.loc["b":"e"])
```

Includes 'e'

Example: Slice with step size

```
print(s[::2])
```

Every 2nd element

Example: Slice from start till given label

```
print(s.loc[: "c"])
```

Example: Slice from given position till end

```
print(s[2:])
```

From 3rd element onwards

Example: Slice using negative positions

```
print(s[-3:])
```

Last 3 elements

Example: Conditional slicing (values greater than 250)

```
subset = s[s > 250]  
print(subset)
```

Series Attributes

- A Series in pandas is a 1D labeled array, and it comes with metadata attributes that describe its structure and contents.
- Unlike methods (which perform actions), attributes are properties that give information about the Series.
- The most useful attributes are:
 - `.index` → Returns the index (row labels).
 - `.values` → Returns the data values as a NumPy array.
 - `.dtype` → Returns the data type of the elements.
 - `.shape` → Returns the shape (length) of the Series.
 - `.name` → Returns or sets the name of the Series.

.index Attribute

- Gives the Index object of the Series.
- Defines labels for rows.
- Immutable (you can't modify directly but can reassign a new Index).
- Useful for iteration, alignment, reindexing, etc.

Example

```
import pandas as pd
import numpy as np
```

```
s = pd.Series([10, 20, 30, 40, 50], index=["a", "b", "c", "d", "e"], name="Marks")
```

Example: Get the index

```
print(s.index)
```

Example: Check index type

```
print(type(s.index))
```

Example: Index length

```
print(len(s.index))
```

Example: Access first index label

```
print(s.index[0])
```

Example: Check if label exists

```
print("c" in s.index)
```

Example: Convert index to list

```
print(list(s.index))
```

Example: Reversing index

```
print(s.index[::-1])
```

Example: Changing index labels

```
s.index = ["p", "q", "r", "s", "t"]  
print(s.index)
```

Example: Using .index.name

```
s.index.name = "Student_ID"  
print(s.index)
```

Example: Numeric index auto-assigned

```
s2 = pd.Series([1, 2, 3])  
print(s2.index)
```

.values Attribute

- Returns underlying NumPy array of values.
- Data types are carried from Series to array.
- Changes in .values don't always reflect back in Series (copy vs view).

Example

```
s = pd.Series([10, 20, 30, 40, 50], index=list("abcde"), name="Marks")
```

Example: Get all values

```
print(s.values)
```

Example: Type of .values

```
print(type(s.values))
```

Example: Convert to list

```
print(s.values.tolist())
```

Example: Access first element

```
print(s.values[0])
```

Example: Perform NumPy operations

```
print(s.values * 2)
```

Example: Boolean condition on values

```
print(s.values > 25)
```

Example: Sum using NumPy array

```
print(s.values.sum())
```


Example: Reverse values

```
print(s.values[::-1])
```

Example: Get dtype from .values

```
print(s.values.dtype)
```

Example: Mixed types

```
s2 = pd.Series([1, "hello", 3.5])  
print(s2.values)
```

.dtype Attribute

- Returns data type of Series values.
- Helps in ensuring consistent types.
- Can be changed with .astype().

Example

```
s = pd.Series([10, 20, 30])
```

Example: Check dtype

```
print(s.dtype)
```

Example: Float Series

```
s2 = pd.Series([1.1, 2.2, 3.3])  
print(s2.dtype)
```

Example: String Series

```
s3 = pd.Series(["a", "b", "c"])  
print(s3.dtype)
```

Example: Boolean Series

```
s4 = pd.Series([True, False, True])  
print(s4.dtype)
```

Example: Mixed types → object

```
s5 = pd.Series([1, "x", 3.5])  
print(s5.dtype)
```

Example: Explicit dtype in constructor

```
s6 = pd.Series([1, 2, 3], dtype="float32")  
print(s6.dtype)
```

Example: Change dtype with astype

```
print(s.astype("float64").dtype)
```

Example: Datetime dtype

```
s7 = pd.Series(pd.date_range("2025-01-01", periods=3))  
print(s7.dtype)
```

Example: Categorical dtype

```
s8 = pd.Series(["low", "medium", "high"], dtype="category")  
print(s8.dtype)
```

Example: Complex numbers

```
s9 = pd.Series([1+2j, 3+4j])  
print(s9.dtype)
```

.shape Attribute

- Returns a tuple showing Series dimensions.
- Since Series is 1D, it always returns (n,).
- Useful for checking length in matrix-like notation.

Example

```
s = pd.Series([10, 20, 30, 40, 50])
```

Example: Get shape

```
print(s.shape)
```

Example: Length check with len()

```
print(len(s)) # Equivalent
```

Example: Empty Series

```
s2 = pd.Series([])  
print(s2.shape)
```

Example: Single-element Series

```
s3 = pd.Series([99])  
print(s3.shape)
```

Example: Numeric with labels

```
s4 = pd.Series([100, 200, 300], index=["x", "y", "z"])  
print(s4.shape)
```

Example: Mixed Series

```
s5 = pd.Series([1, "hi", 3.14, True])  
print(s5.shape)
```

Example: Range of 1000

```
s6 = pd.Series(range(1000))  
print(s6.shape)
```

Example: Numpy array inside Series

```
arr = np.arange(20)  
s7 = pd.Series(arr)  
print(s7.shape)
```

Example: Reshape attempt (not possible directly)

```
print(s.shape[0]) # Get length
```

Example: Compare Series lengths

```
s8 = pd.Series([1, 2, 3, 4, 5, 6])  
print(s.shape, s8.shape)
```

.name Attribute

- A descriptive label for the Series.
- Acts like a column name when Series becomes part of a DataFrame.
- Can be set, modified, or removed (None).

Example

```
s = pd.Series([100, 200, 300], index=["a", "b", "c"])
```

Example: Default name

```
print(s.name)
```

Example: Assign a name

```
s.name = "Scores"  
print(s.name)
```

Example: Change the name

```
s.name = "Student Marks"  
print(s.name)
```

Example: Remove name

```
s.name = None  
print(s.name)
```

Example: Name at creation

```
s2 = pd.Series([1, 2, 3], name="Population")  
print(s2.name)
```

Example: Series as DataFrame column

```
df = s2.to_frame()
print(df)
```

Example: Using name in plotting

```
s2.plot(title=s2.name)
```

Example: Copy Series with new name

```
s3 = s2.rename("People")
print(s3.name)
```

Example: Multiple Series with different names

```
s4 = pd.Series([4, 5, 6], name="X")
s5 = pd.Series([7, 8, 9], name="Y")
df2 = pd.concat([s4, s5], axis=1)
print(df2)
```

Example: Apply function and keep name

```
s6 = s2.apply(lambda x: x*2)
s6.name = s2.name
print(s6.name)
```

Pandas Series Attributes – Comparison Table

Attribute	Purpose / Description	Usage Example	Return Type	Key Notes
<code>.index</code>	Returns the row labels (Index object) of the Series	<code>s.index</code>	<code>pandas.Index</code>	Immutable (cannot modify directly); can assign a new index; supports names
<code>.values</code>	Returns the underlying NumPy array of values	<code>s.values</code>	<code>numpy.ndarray</code>	Can be converted to list; dtype depends on Series values; may copy or view
<code>.dtype</code>	Returns the data type of elements in Series	<code>s.dtype</code>	<code>numpy.dtype</code> / <code>pandas type</code>	Useful for type checking; can be changed using <code>.astype()</code>
<code>.shape</code>	Returns the dimension (length) of Series as a tuple	<code>s.shape</code>	tuple	Always (n,) since Series is 1D; equivalent to <code>len(s)</code>
<code>.name</code>	Returns or sets the label (name) of the Series	<code>s.name</code>	str or None	Acts as a column name when converted to DataFrame; can rename

Example for All Attributes

```
import pandas as pd
s = pd.Series([10, 20, 30], index=["a", "b", "c"], name="Marks")
print("Index :", s.index)           # Index(['a','b','c'], dtype='object')
print("Values:", s.values)          # [10 20 30]
print("Dtype :", s.dtype)           # int64
print("Shape :", s.shape)           # (3,)
print("Name :", s.name)             # Marks
```

Series Operations

- A pandas Series is built on top of NumPy arrays but enhanced with index labels.
- This makes Series operations very powerful, because you can do arithmetic, comparisons, aggregations, etc., just like with arrays, but with the added benefit of labels.
- Operations are vectorized (apply to all elements at once, no loop needed).
- Series automatically align values based on index labels when operating with another Series.
- Supports broadcasting, meaning scalar values are extended across the Series during operations.
- Operations preserve index labels, making results more interpretable.

Vectorized Operations

- Vectorized operations mean performing computations on **entire arrays/Series at once** instead of looping through elements.
- They are much **faster** because they use optimized C-level routines under the hood.
- Examples: **+**, **-**, *****, **/**, comparisons, mathematical functions (np.sqrt, np.exp, etc.).

Example

```
import pandas as pd
import numpy as np
```

```
s = pd.Series([10, 20, 30, 40, 50], index=list("abcde"))
```

Example: Add scalar to Series

```
print(s + 5)
```

Example: Subtract scalar

```
print(s - 10)
```

Example: Multiply by scalar

```
print(s * 2)
```

Example: Divide by scalar

```
print(s / 2)
```

Example: Power operation

```
print(s ** 2)
```

Example: Comparison (greater than)

```
print(s > 25)
```

Example: Modulus operation

```
print(s % 3)
```

Example: Apply NumPy function (sqrt)

```
print(np.sqrt(s))
```

Example: Apply exponential

```
print(np.exp(s))
```

Example: Chain operations

```
print((s * 2) + 100)
```

Alignment in Series

- When two Series with **different indexes** are used in operations, pandas **aligns values by labels**.
- If a label is missing in one Series, result is NaN.
- This makes operations safer and more intuitive.

Example

```
s1 = pd.Series([10, 20, 30], index=["a", "b", "c"])  
s2 = pd.Series([100, 200, 300], index=["b", "c", "d"])
```

Example: Arithmetic Operations with alignment

```
print(s1 + s2)  
print(s1 - s2)  
print(s1 * s2)  
print(s1 / s2)
```

Example: Index mismatch result (NaN)

```
print(s1 + pd.Series([5, 10], index=["x", "y"]))
```

Example: Fill missing values before operation

```
print((s1 + s2).fillna(0))
```

Example: Align explicitly with add()

```
print(s1.add(s2, fill_value=0))
```

Example: Align explicitly with sub()

```
print(s1.sub(s2, fill_value=0))
```

Example: Align explicitly with mul()

```
print(s1.mul(s2, fill_value=1))
```

Example: Align explicitly with div()

```
print(s1.div(s2, fill_value=1))
```

Broadcasting in Series

- **Broadcasting** means applying an operation between a **Series and a scalar**, where the scalar is **extended** to match the length of the Series.
- This avoids manual looping and makes operations concise.
- Broadcasting works with arithmetic and comparisons.

Example

```
s = pd.Series([10, 20, 30, 40, 50], index=list("abcde"))
```

Example: Arithmetic scalar

```
print(s + 10)
print(s - 5)
print(s * 3)
print(s / 10)
```

Example: Compare with scalar

```
print(s > 25)
```

Example: Less than comparison

```
print(s < 35)
```

Example: Equality comparison

Example: Modulus operation

```
print(s % 7)
```

Example: Square root broadcasting

```
print(np.sqrt(s + 100))          # broadcasting inside function
print(s == 20)
```

Example: Logical operation with scalar

```
print(s >= 30)
```

Points to Note

- **Vectorized operations:** Perform element-wise operations on entire Series at once (fast and concise).
- **Alignment:** When two Series are combined, pandas automatically matches values by index labels (missing labels → NaN).
- **Broadcasting:** Scalars are applied across all elements in Series automatically.

Series Methods

A pandas.Series is a 1-D labeled array. Its methods fall into a few practical families:

1. **Introspection & basic views:** Shape/peek/structure: `.head()`, `.tail()`, `.sample()`, `.info()` (via `DataFrame`), `.describe()`, `.value_counts()`, `.unique()`, `.nunique()`.
2. **Index & alignment:** Reindex/rename/sort: `.reindex()`, `.rename()`, `.sort_index()`, `.sort_values()`.
3. **Computation & stats:** Aggregations: `.sum()`, `.mean()`, `.median()`, `.min()`, `.max()`, `.std()`, `.var()`, `.quantile()`.
4. **Elementwise transforms:** `.map()`, `.apply()`, `.round()`, string ops via `.str`, datetime ops via `.dt`, categorical ops.
5. **Missing data handling:** `.isna()`, `.notna()`, `.fillna()`, `.dropna()`.
6. **Type & casting:** `.astype()`, `.to_numpy()`, `.to_list()`.
7. **Combining:** `.append()` (deprecated), `pd.concat([...], axis=0)`, arithmetic with alignment, comparison ops.
8. **Selection & filtering helpers:** `.where()`, `.mask()`, `.clip()`, `.between()`.

Introspection & Basic Views in Pandas

- Introspection means examining the internal structure and metadata of a Pandas object (like a Series or DataFrame) to understand what it contains.
- It helps us quickly check data types, shape, index, and a preview of data.
- For a Series, common introspection methods and attributes are:
 - `.head(n)` → shows the first n rows (default = 5).
 - `.tail(n)` → shows the last n rows (default = 5).
 - `.sample(n)` → shows a random sample of n rows.
 - `.describe()` → summary statistics.
 - `.info()` → detailed structure.

These methods are used for basic views and quick exploration.

`.head()` in Pandas

- `.head(n)` returns the first n elements of a Series.
- Default: n = 5.

Examples of `.head()`

```
import pandas as pd

# Example Series
s = pd.Series([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
# Default (first 5 rows)
print(s.head())
# First 3 rows
print(s.head(3))
# Larger than length (no error)
print(s.head(20))
# On a string series
s2 = pd.Series(["Apple", "Banana", "Mango", "Orange", "Grapes"])
print(s2.head(2))
# On a Series with custom index
s3 = pd.Series([100, 200, 300, 400], index=["a", "b", "c", "d"])
print(s3.head(2))
# On a Series with missing values
s4 = pd.Series([1, None, 3, None, 5, 6])
print(s4.head())
# On a Series of boolean values
s5 = pd.Series([True, False, True, True, False])
print(s5.head(3))
# On a large range Series
s6 = pd.Series(range(1, 101))
print(s6.head())
# Using inside DataFrame column
df = pd.DataFrame({"Marks": [50, 60, 70, 80, 90, 100]})
print(df["Marks"].head(4))
```

.tail() in Pandas

- .tail(n) returns the last n elements of a Series.
- Default: n = 5.

Examples of .tail()

```
# Default (last 5 rows)
s = pd.Series([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
print(s.tail())

# Last 3 rows
print(s.tail(3))

# Larger than length (no error)
print(s.tail(20))

# On string series
s2 = pd.Series(["Apple", "Banana", "Mango", "Orange", "Grapes"])
print(s2.tail(2))

# On custom index series
s3 = pd.Series([100, 200, 300, 400], index=["a", "b", "c", "d"])
print(s3.tail(2))

# On series with missing values
s4 = pd.Series([1, None, 3, None, 5, 6])
print(s4.tail(4))

# On boolean series
s5 = pd.Series([True, False, True, True, False])
print(s5.tail(2))

# On large range
s6 = pd.Series(range(1, 101))
print(s6.tail(6))

# On DataFrame column
df = pd.DataFrame({"Marks": [50, 60, 70, 80, 90, 100]})
print(df["Marks"].tail(3))
```

.sample() in Pandas

- .sample(n) returns n random rows from the Series.
- If n is not given, returns 1 random element.

Examples of .sample()

```
# One random element (default)
s = pd.Series([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
print(s.sample())

# 3 random elements
print(s.sample(3))

# Sample with fraction (20% of data)
print(s.sample(frac=0.2))
```

```
# Sample with replacement
print(s.sample(5, replace=True))
    If replace=True, then the same element could be picked multiple times.
    If replace=False then you cannot pick the same item more than once (no repetition).

# On string series
s2 = pd.Series(["Apple", "Banana", "Mango", "Orange", "Grapes"])
print(s2.sample(2))

# On custom index
s3 = pd.Series([100, 200, 300, 400], index=["a", "b", "c", "d"])
print(s3.sample(1))

# On series with NaN values
s4 = pd.Series([1, None, 3, None, 5, 6])
print(s4.sample(2))

# On boolean series
s5 = pd.Series([True, False, True, True, False])
print(s5.sample(3))

# On large range (random 10 numbers)
s6 = pd.Series(range(1, 101))
print(s6.sample(10))

# On DataFrame column
df = pd.DataFrame({"Marks": [50, 60, 70, 80, 90, 100]})
print(df["Marks"].sample(2))
```

.describe() on Series in Pandas

- `.describe()` when called on a Series returns a summary of statistics:
 - **For numeric Series** → count, mean, std, min, 25%, 50% (median), 75%, max
 - **count**: Number of non-null (non-NaN) values in the Series, ignores missing values (NaN)
 - **mean**: The average of all values.
 - **std (standard deviation)**: Shows how spread out the values are from the mean.
 - ✓ Low std → values are close to mean,
 - ✓ High std → values are spread out.
 - **min**: The smallest value in the series
 - **25% (1st Quartile / Q1)**: The value below which 25% of data lies. Also called first quartile (Q1).
 - **50% (Median / Q2)**: The middle value when data is sorted. 50% of values lie below, 50% above.
 - **75% (3rd Quartile / Q3)**: The value below which 75% of data lies. Also called third quartile (Q3).
 - **Max**: The largest value in the Series.
 - **For object/string Series** → count, unique, top (most frequent), freq (frequency of top)
 - **unique**: The number of distinct values in the Series.
 - **top (most frequent)**: The most common (mode) value in the Series.
 - **freq (frequency of top)**: The number of times the top value occurs.
 - **For boolean Series** → count, unique, top (True/False), freq
 - **top**: whichever value (True or False) appears most frequently
 - **freq**: how many times that top value appears
- Used to summarize Series data distribution quickly.

Examples of .describe() on Series

```
import pandas as pd
import numpy as np

# Numeric Series
s1 = pd.Series([10, 20, 30, 40, 50])
print(s1.describe())

# Numeric Series with more values
s2 = pd.Series([5, 15, 25, 35, 45, 55, 65])
print(s2.describe())

# Series with floats
s3 = pd.Series([1.2, 3.5, 2.8, 9.1, 7.3])
print(s3.describe())

# String Series
s4 = pd.Series(["apple", "banana", "apple", "orange", "banana", "apple"])
print(s4.describe())

# Boolean Series
s5 = pd.Series([True, False, True, True, False, True])
print(s5.describe())

# Series with missing values
s6 = pd.Series([10, 20, np.nan, 40, np.nan, 60])
print(s6.describe())

# Large random Series
s7 = pd.Series(np.random.randint(1, 100, 20))
print(s7.describe())

# Series with custom percentiles
print(s7.describe(percentiles=[0.1, 0.9]))

# Categorical string values
s8 = pd.Series(["HR", "IT", "Finance", "IT", "HR", "HR"])
print(s8.describe())

# Series with dates (object type)
s9 = pd.Series(pd.date_range("2024-01-01", periods=5))
print(s9.describe())
```

.info() on Series in Pandas

- .info() gives a summary about the Series: Index range, Number of non-null entries, Data type (dtype), Memory usage.
- Helps to check data structure, missing values, and memory footprint.

Examples of .info() on Series

```
# Simple numeric Series
s1 = pd.Series([10, 20, 30, 40, 50])
print(s1.info())
```

```

# String Series
s2 = pd.Series(["a", "b", "c", "d"])
print(s2.info())

# Boolean Series
s3 = pd.Series([True, False, True, True])
print(s3.info())

# Series with missing values
s4 = pd.Series([1, None, 3, None, 5])
print(s4.info())

# Series with custom index
s5 = pd.Series([100, 200, 300], index=["x", "y", "z"])
print(s5.info())

# Series with floats
s6 = pd.Series([1.1, 2.2, 3.3, 4.4])
print(s6.info())

# Series with large random integers
s7 = pd.Series(np.random.randint(1, 1000, 100))
print(s7.info())

# Series with datetime values
s8 = pd.Series(pd.date_range("2024-01-01", periods=10))
print(s8.info())

# Series with categorical values
s9 = pd.Series(pd.Categorical(["small", "medium", "large", "medium"]))
print(s9.info())

# Empty Series
s10 = pd.Series([], dtype="float64")
print(s10.info())

```

.value_counts()

- Returns a Series containing counts of unique values in descending order.
- Useful to see frequency distribution.
- Parameters:
 - **normalize=True** → gives relative frequencies (percentages).
 - **ascending=True** → sort in ascending order.
 - **dropna=False** → include NaN in the counts.

Example: Numbers

```

import pandas as pd
s = pd.Series([10, 20, 10, 30, 20, 10])
print(s.value_counts())

```

Example: Strings

```

s = pd.Series(["apple", "banana", "apple", "mango", "banana", "apple"])
print(s.value_counts())

```

Example: Boolean

```
s = pd.Series([True, False, True, True, False])  
print(s.value_counts())
```

Example: With normalize=True

```
s = pd.Series([1, 1, 2, 2, 2, 3])  
print(s.value_counts(normalize=True))
```

.unique()

- Returns a NumPy array of unique values in the order they appear.
- Does not sort the values.

Example: Numbers

```
s = pd.Series([10, 20, 10, 30, 20])  
print(s.unique())
```

Example: Strings

```
s = pd.Series(["apple", "banana", "apple", "mango"])  
print(s.unique())
```

Example: Boolean

```
s = pd.Series([True, False, True, True])  
print(s.unique())
```

Example: Mixed Types

```
s = pd.Series([1, "apple", 1, "banana", True])  
print(s.unique())
```

.nunique()

- Returns the number of unique values (count of distinct values).
- Parameters:
 - **dropna=True (default)** → ignores NaN.
 - **dropna=False** → counts NaN as unique.

Example: Numbers

```
s = pd.Series([10, 20, 10, 30])  
print(s.nunique())
```

Example: Strings

```
s = pd.Series(["apple", "banana", "apple", "mango"])  
print(s.nunique())
```

Example: With NaN

```
import numpy as np
s = pd.Series([1, 2, 2, np.nan, 3, np.nan])
print(s.nunique()) # default dropna=True
print(s.nunique(dropna=False))
```

Example: Boolean

```
s = pd.Series([True, False, True, True])
print(s.nunique())
```

DataFrame in Pandas

- A DataFrame is like a table (like Excel or SQL table) in Python.
- It has rows and columns.
- Each column can have different types of data (numbers, text, dates, etc.).
- The row labels are called index, and the column labels are the column names.

Example

```
import pandas as pd
data = {
    'Name': ['Amit', 'Neha', 'Raj'],
    'Age': [21, 22, 23],
    'City': ['Agra', 'Delhi', 'Mumbai']
}
df = pd.DataFrame(data)
print(df)
```

Creating DataFrames from Dictionaries

- A dictionary in Python is a set of key:value pairs.
- In pandas, when you create a DataFrame from a dictionary:
 - **Keys** → Column names
 - **Values** → Data inside the columns
- Values can be lists, arrays, Series, scalars, or even another dictionary.
- If values are lists/arrays, they should usually be of the same length.
- If values are dicts/Series, pandas aligns them by their index/keys.
- You can also:
 - Set your own row labels (index).
 - Control column order.
 - Use nested dictionaries.
 - Have scalar values, which pandas repeats for each row.

Example: Dictionary of Lists (basic)

```
import pandas as pd
data = {
    "Name": ["Asha", "Rahul", "Karan"],
    "Age": [21, 22, 20],
    "City": ["Agra", "Delhi", "Jaipur"]
}
df = pd.DataFrame(data)
print(df)
```

Each key becomes a column, and list values fill the rows.

Example: Dict of Lists with custom column order

```
data = {
    "Age": [21, 22, 20],
    "Name": ["Asha", "Rahul", "Karan"],
    "City": ["Agra", "Delhi", "Jaipur"]
}

df = pd.DataFrame(data, columns=["Name", "Age", "City"]) # Order fixed
print(df)
```

You can control how columns appear.

Example: Dictionary of NumPy Arrays

```
import numpy as np
data = {
    "X": np.array([1, 2, 3]),
    "Y": np.array([10, 20, 30])
}

df = pd.DataFrame(data)
print(df)
```

Arrays work just like lists.

Example: Dictionary of Series (aligned by index)

```
s1 = pd.Series([85, 90, 88], index=["Math", "Eng", "Sci"])
s2 = pd.Series([78, 82, 91], index=["Math", "Eng", "Sci"])

df = pd.DataFrame({"Asha": s1, "Rahul": s2})
print(df)
```

Index becomes the row labels, values align automatically.

Example: Dictionary of Dictionaries

```
data = {
    "Asha": {"Math": 85, "Eng": 90},
    "Rahul": {"Math": 78, "Sci": 91}
}

df = pd.DataFrame(data)
print(df)
```

Outer keys → columns, inner keys → row index.

Example: Dictionary of Dictionaries with orient='index'

```
data = {
    "Asha": {"Math": 85, "Eng": 90},
    "Rahul": {"Math": 78, "Sci": 91}
}

df = pd.DataFrame.from_dict(data, orient="index")
print(df)
```

Outer keys become row labels instead of columns.

Example: Dictionary of scalars (repeated values)

```
data = {"Course": "Data Analytics", "Institute": "Learn2Earn Labs"}
df = pd.DataFrame(data, index=[101, 102, 103])
print(df)
```

Scalar values repeat for all rows.

Example: Dictionary of Lists with custom index

```
data = {"Name": ["Asha", "Rahul", "Karan"], "Score": [85, 90, 88]}
df = pd.DataFrame(data, index=["S1", "S2", "S3"])
print(df)
```

We control row labels using index.

Example: Multi-level column labels

```
data = {
    ("Marks", "Math"): [85, 78, 92],
    ("Marks", "Eng"): [90, 82, 88]
}
df = pd.DataFrame(data)
print(df)
print(df.columns)                                # MultiIndex
```

Column names can be tuples, creating multi-level columns.

Example: Rename Columns

```
data = {"Student Name": ["Asha", "Rahul"], "Total Marks": [175, 160]}
df = pd.DataFrame(data)
df = df.rename(columns={"Student Name": "Name", "Total Marks": "Total"})
print(df)
```

After creating DataFrame, you can clean up column names.

Example: Mixed Data Types

```
data = [
    {"id": 1, "val": 10.5},
    {"id": 2, "val": "N/A"},
    {"id": 3, "val": None}
]

df = pd.DataFrame(data)
print(df)
print(df.dtypes)
```

A column can hold numbers + strings + None.

Example: Nested Dictionary → json_normalize

```
data = [
    {"id": 1, "user": {"name": "Asha", "city": "Agra"}},
    {"id": 2, "user": {"name": "Rahul", "city": "Delhi"}}
]

df = pd.json_normalize(data)
print(df)
```

Flattens nested JSON-like structure.

Example: Nested List Inside Dict → json_normalize with record_path

```
data = {
    "batch": "A",
    "students": [
        {"roll": 1, "name": "Asha"},
        {"roll": 2, "name": "Rahul"}
    ]
}

df = pd.json_normalize(data, record_path=["students"], meta=["batch"])
print(df)
```

Expands a list inside dictionary into rows.

Example: Choose only some columns

```
data = [
    {"name": "Asha", "age": 21, "city": "Agra"},
    {"name": "Rahul", "age": 22, "city": "Delhi"}
]

df = pd.DataFrame(data, columns=["name", "city"]) # drop "age"
print(df)
```

Creating DataFrames from NumPy Arrays

- A NumPy array is a fast, table-like structure (mostly numbers).
- When you convert it into a DataFrame:
 - A **2D array** → full table (rows × columns)
 - A **1D array** → single column
- By default, pandas will give numeric column names (0,1,2 ...) and row indexes (0,1,2 ...).
- You can:
 - Add custom column names.
 - Add custom row indexes.
 - Convert structured arrays (with named fields) → directly into labeled DataFrames.
 - Use random data or reshaping before creating the DataFrame.

Example: 2D Array with default labels

```
import pandas as pd
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6]])

df = pd.DataFrame(arr)
print(df)
```

Columns and rows are numbered automatically.

Example: 2D Array with custom column names

```
arr = np.array([[85, 90],
                [78, 82],
                [92, 88]])

df = pd.DataFrame(arr, columns=["Math", "English"])
print(df)
```

Column names are meaningful.

Example: 2D Array with custom row index + columns

```
arr = np.array([[85, 90],
                [78, 82],
                [92, 88]])

df = pd.DataFrame(arr, index=["Asha", "Rahul", "Karan"], columns=["Math", "Eng"])
print(df)
```

Both row labels and column names set.

Example: 1D Array → single column dataframe

```
arr = np.array([10, 20, 30])

df = pd.DataFrame(arr, columns=["Value"])
print(df)
```

Converts 1D data into one column.

Example: Structured array (with named fields)

```
arr = np.array([(1, "Asha", 85.0), (2, "Rahul", 90.5)],
               dtype=[("id", "i4"), ("name", "U10"), ("score", "f4")])

df = pd.DataFrame(arr)
print(df)
```

Field names become column headers.

Example: Random numbers with column headers

```
arr = np.random.randint(50, 101, size=(4, 3))

df = pd.DataFrame(arr, columns=["Math", "Eng", "Sci"])
print(df)
```

Quickly generate sample datasets.

Example: Record Array to dataframe

```
r = np.rec.array([(1, "Asha"), (2, "Rahul")],
                 dtype=[("id", "i4"), ("name", "U10")])

df = pd.DataFrame(r)
print(df)
```

Works like structured arrays.

Example: Mixed types with object dtype

```
arr = np.array([[1, "Asha"], [2, "Rahul"]], dtype=object)

df = pd.DataFrame(arr, columns=["ID", "Name"])
print(df)
```

Object dtype lets you mix numbers + text.

Example: Multi-level columns from tuples

```
arr = np.array([[85, 90, 78, 82],
                [92, 88, 80, 84]])

cols = pd.MultiIndex.from_tuples([
    ("Asha", "Math"),
    ("Asha", "Eng"),
    ("Rahul", "Math"),
    ("Rahul", "Eng")
])
df = pd.DataFrame(arr, columns=cols)
print(df)
```

Creates hierarchical column labels.

Example: Reshape array before dataframe

```
arr = np.arange(1, 7)                # 1D → 6 elements
reshaped = arr.reshape(2, 3)         # convert to 2×3
df = pd.DataFrame(reshaped, columns=["C1", "C2", "C3"])
print(df)
```

Reshape arrays into table form before DataFrame.

Creating DataFrames from Series

- A Series is like a single column of data with labels (index).
- When converting Series into a DataFrame:
 - **Single Series** → one column DataFrame.
 - **Multiple Series** → combined side-by-side (columns).
- If Series have different indexes, pandas aligns them → missing values become NaN.
- You can:
 - Rename the Series to set the column name.
 - Use `.to_frame()` to directly convert Series.
 - Transpose (`.T`) a Series to make it a row instead of a column.
 - Expand Series of lists into multiple columns.
 - Stack/unstack Series with MultiIndex into tabular form.

Example: Single Series → one column dataframe

```
s = pd.Series([85, 90, 88], index=["Asha", "Rahul", "Karan"], name="Score")
df = s.to_frame()
print(df)
```

Converts a Series into a DataFrame with one column.

Example: Rename series column

```
s = pd.Series([100, 200, 150], name="Amount")
df = s.to_frame(name="Amount_Rupees")
print(df)
```

Change the column name while creating DataFrame.

Example: Two series combined → side-by-side columns

```
s1 = pd.Series({"Asha": 85, "Rahul": 90, "Karan": 88}, name="Math")
s2 = pd.Series({"Asha": 90, "Rahul": 82, "Karan": 88}, name="Eng")
df = pd.concat([s1, s2], axis=1)
print(df)
```

Each Series becomes a column.

Example: Different indexes align automatically

```
s1 = pd.Series({"Asha": 85, "Rahul": 90}, name="Math")
s2 = pd.Series({"Rahul": 82, "Karan": 88}, name="Eng")
df = pd.concat([s1, s2], axis=1)
print(df)
```

Index is the union of both Series.

Example: Dict of series → columns from keys

```
math = pd.Series([85, 90, 88], index=["Asha", "Rahul", "Karan"])
eng = pd.Series([90, 82, 88], index=["Asha", "Rahul", "Karan"])
df = pd.DataFrame({"Math": math, "Eng": eng})
print(df)
```

Dictionary key becomes column name.

Example: Series → single row via transpose

```
s = pd.Series({"Math": 85, "Eng": 90, "Sci": 88}, name="Asha")
df = s.to_frame().T
print(df)
```

.T flips Series to row.

Example: Multiple series with custom index order

```
s1 = pd.Series([85, 90, 88], index=["Asha", "Rahul", "Karan"], name="Math")
s2 = pd.Series([90, 82, 88], index=["Asha", "Rahul", "Karan"], name="Eng")
df = pd.concat([s1, s2], axis=1).reindex(index=["Karan", "Asha", "Rahul"])
print(df)
```

Rearranges row order after combining.

Example: Series with multiindex → unstack to table

```
s = pd.Series([85, 90, 78, 82],
              index=pd.MultiIndex.from_product([["Asha", "Rahul"], ["Math", "Eng"]])
)
df = s.unstack()
print(df)
```

MultiIndex expands into columns.

Example: Series of lists → expand into multiple columns

```
s = pd.Series([[85, 90], [78, 82], [92, 88]],
              index=["Asha", "Rahul", "Karan"], name="Marks")
df = pd.DataFrame(s.tolist(), index=s.index, columns=["Math", "Eng"])
print(df)
```

Each list inside Series expands into two columns.

Example: Use series inside dict → normal dataframe

```
names = pd.Series(["Asha", "Rahul", "Karan"])
scores = pd.Series([85, 90, 88])
df = pd.DataFrame({"Name": names, "Score": scores})
print(df)
```

Combine multiple Series like columns in a dictionary.

Creating DataFrames from CSV / Excel Files

- Many times, data is stored in external files like CSV or Excel.
- pandas provides two powerful functions:
 - `pd.read_csv("file.csv")` → read CSV (Comma Separated Values) files.
 - `pd.read_excel("file.xlsx")` → read Excel files.
- CSV files are plain text (fast, lightweight).
- Excel files can have multiple sheets, formatting, formulas.
- By default:
 - First row = column names.
 - Row numbers = index (0,1,2 ...).
- You can:
 - Choose specific columns (usecols).
 - Set a column as index (index_col).
 - Handle missing headers (header=None).
 - Parse dates automatically.
 - Limit rows (nrows), skip rows (skiprows).
 - Load specific Excel sheet (sheet_name).

students.csv

```
Roll,Name,Age,City,Score
101,Asha,21,Agra,85
102,Rahul,22,Delhi,90
103,Karan,20,Jaipur,88
104,Neha,21,Lucknow,92
105,Anil,23,Kanpur,80
```

Example: Basic csv file

```
df = pd.read_csv("students.csv")
print(df.head())
```

Reads the CSV into a DataFrame (first 5 rows).

Example: csv with a column as index

```
df = pd.read_csv("students.csv", index_col="Roll")
print(df.head())
```

Uses Roll column as row labels.

Example: Select only some columns

```
df = pd.read_csv("students.csv", usecols=["Name", "City", "Score"])
print(df.head())
```

Reads only the required columns.

Example: Specify data types

```
df = pd.read_csv("students.csv", dtype={"Roll": "int64", "Name": "string"})
print(df.dtypes)
```

Forces correct data type.

attendance.csv

```
Roll,Name,Date,Status
101,Asha,2025-08-01,Present
102,Rahul,2025-08-01,Absent
103,Karan,2025-08-01,Present
101,Asha,2025-08-02,Present
102,Rahul,2025-08-02,Present
103,Karan,2025-08-02,Absent
```

Example: Parse dates during load

```
df = pd.read_csv("attendance.csv", parse_dates=["Date"])
print(df.dtypes)
```

Converts string dates into real datetime objects.

marks_no_header.csv

```
101,85,90
102,78,82
103,92,88
104,88,84
```

Example: File without headers → supply column names

```
df = pd.read_csv("marks_no_header.csv", header=None,
                 names=["Roll", "Math", "Eng"])
print(df.head())
```

Useful when file has no column names.

report.csv

```
# Student Performance Report
# Generated by Learn2Earn Labs on 2025-08-20
Roll,Name,Math,Eng,Sci
101,Asha,85,90,88
102,Rahul,78,82,91
103,Karan,92,88,85
104,Neha,88,84,89
```

Example: Skip rows (e.g., metadata lines)

```
df = pd.read_csv("report.csv", skiprows=2)           # skip first 2 lines
print(df.head())
```

Ignore unwanted rows at the top.

data.csv

```
Roll,Name,Age,City,Score
101,Asha,21,Agra,85
102,Rahul,22,Delhi,90
103,Karan,20,Jaipur,88
104,Neha,21,Lucknow,92
105,Anil,23,Kanpur,80
106,Pooja,22,Meerut,89
107,Arjun,24,Varanasi,76
108,Sneha,21,Noida,95
109,Rohit,22,Gurgaon,84
110,Meera,23,Chennai,91
111,Amit,25,Pune,78
112,Divya,22,Bangalore,87
```

Example: Limit number of rows (preview)

```
df = pd.read_csv("data.csv", nrows=5)
print(df)
```

Reads only first 5 rows from a large file.

students.xlsx

- **Sheet 1: Students (default sheet)**

Roll	Name	Age	City	Score
101	Asha	21	Agra	85
102	Rahul	22	Delhi	90
103	Karan	20	Jaipur	88
104	Neha	21	Lucknow	92
105	Anil	23	Kanpur	80

- **Sheet 2: Marks**

Roll	Math	Eng	Sci
101	85	90	88
102	78	82	91
103	92	88	85
104	88	84	89
105	80	79	87

- **Sheet 3: Attendance**

Roll	Name	Date	Status
101	Asha	2025-08-01	Present
102	Rahul	2025-08-01	Absent
103	Karan	2025-08-01	Present
101	Asha	2025-08-02	Present
102	Rahul	2025-08-02	Present
103	Karan	2025-08-02	Absent

Example: Basic excel file (default first sheet)

```
df = pd.read_excel("students.xlsx")          # requires openpyxl installed
print(df.head())
```

Reads Excel file into DataFrame.

Example: Excel with Specific Sheet + Index Column

```
df = pd.read_excel("students.xlsx", sheet_name="Marks", index_col="Roll")
print(df.head())
```

Loads data from specific sheet, sets index column.

Example: Read all sheets into DataFrames

```
sheets = pd.read_excel("students.xlsx", sheet_name=None)
# Create DataFrames from each sheet
df_students = pd.DataFrame(sheets["Students"])
df_marks = pd.DataFrame(sheets["Marks"])
df_attendance = pd.DataFrame(sheets["Attendance"])
print("Students DataFrame:\n", df_students.head(), "\n")
print("Marks DataFrame:\n", df_marks.head(), "\n")
print("Attendance DataFrame:\n", df_attendance.head(), "\n")
```

Example: Create a combined student + marks dataframe

merge students and marks using roll

```
df_student_marks = pd.merge(df_students, df_marks, on="Roll")
```

```
print("Student + Marks DataFrame:\n", df_student_marks.head(), "\n")
```

New DataFrame: contains Roll, Name, Age, City, Score, Math, Eng, Sci.

Example: Create an attendance summary dataframe

Count how many days each student was present

```
df_attendance_summary = (  
    df_attendance[df_attendance["Status"] == "Present"]  
    .groupby("Roll")  
    .size()  
    .reset_index(name="Present_Days")  
)
```

```
print("Attendance Summary DataFrame:\n", df_attendance_summary.head(), "\n")
```

New DataFrame: contains Roll, Present_Days.

Example: Create a final combined report dataframe

Merge Marks + Attendance summary

```
df_final_report = pd.merge(df_student_marks, df_attendance_summary, on="Roll",  
    how="left")
```

```
print("Final Report DataFrame:\n", df_final_report.head(), "\n")
```

New DataFrame: has everything: personal details + marks + attendance.

Example: Add a calculated column → average marks dataframe

```
df_final_report["Average"] = df_final_report[["Math", "Eng", "Sci"]].mean(axis=1)
```

```
print("Final Report with Average:\n", df_final_report.head(), "\n")
```

New DataFrame: adds an Average column with mean marks.

DataFrame Attributes

- A DataFrame is a 2D table of data with rows and columns.
- Pandas exposes handy attributes (not functions) to quickly inspect/modify the structure:
 - `.columns` → an Index of column labels (names of the columns).
 - `.index` → an Index of row labels (row identifiers).
 - `.shape` → a (rows, cols) tuple telling how big the table is.
 - `.dtypes` → a Series of data types for each column (e.g., int64, float64, object, string, bool, datetime64[ns], category).
- `.columns` and `.index` are Index objects, which are immutable containers (you can reassign them, but the object itself behaves immutably).
- `.shape` is a plain tuple; `.dtypes` is a Series keyed by column names.

`.columns` — Column Labels

- An Index object listing all column names in order.
- You can read it to inspect names, or assign a new list/Index to rename/reorder columns.

Example: View column labels

```
df = pd.DataFrame({"name": ["Asha", "Rahul"], "age": [21, 22], "city": ["Agra", "Delhi"]})
print(df.columns)
```

Example: Assign a set of column names

```
df = pd.DataFrame({"c1": [1,2], "c2": [3,4], "c3": [5,6]})
df.columns = ["id", "score", "rank"]
print(df.head())
```

Example: Rename via `set_axis` (keeps length)

```
df = pd.DataFrame({"A": [1,2], "B": [3,4]})
df = df.set_axis(["col_A", "col_B"], axis=1)
print(df.columns)
```

Example: Map-like rename with `rename(columns=...)`

```
df = pd.DataFrame({"Student Name": ["Asha", "Rahul"], "Total Marks": [175, 160]})
df = df.rename(columns={"Student Name": "name", "Total Marks": "total"})
print(df.columns)
```

Example: Normalize: lowercase, strip spaces, replace spaces

```
df = pd.DataFrame({"Student Name": ["Asha"], "Total Marks": [175]})
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
print(df.columns) # Index(['student_name', 'total_marks'], dtype='object')
```

Example: Reorder columns using `.columns`

```
df = pd.DataFrame({"name": ["Asha", "Rahul"], "age": [21, 22], "city": ["Agra", "Delhi"]})
new_order = ["city", "name", "age"]
df = df[new_order]
print(df.columns)
```

Example: Add prefix/suffix to all columns

```
df = pd.DataFrame({"math": [85, 78], "eng": [90, 82]})
df = df.add_prefix("sub_").add_suffix("_marks")
print(df.columns)
```

Example: Create MultiIndex columns

```
df = pd.DataFrame(
    np.array([[85, 90, 78, 82]]),

    columns=pd.MultiIndex.from_tuples([("Asha", "math"), ("Asha", "eng"), ("Rahul", "math"), ("Rahul", "eng")])
)
print(df.columns)
```

Example: Insert a new column and check `.columns`

```
df = pd.DataFrame({"name": ["Asha", "Rahul"], "math": [85, 78]})
df.insert(1, "eng", [90, 82])
print(df.columns)
```

Example: Get column names as a plain Python list

```
df = pd.DataFrame({"a": [1], "b": [2], "c": [3]})
col_list = df.columns.tolist()
print(col_list)
```

.index — Row Labels

- An Index object for row labels; default is `RangeIndex(0, 1, 2, ...)`.
- You can set it from a column (IDs/dates), name it, sort by it, reindex, etc.

Example: Default `RangeIndex`

```
df = pd.DataFrame({"name": ["Asha", "Rahul"], "age": [21, 22]})
print(df.index)
```

Example: Set a column as index

```
df = pd.DataFrame({"roll": [101, 102], "name": ["Asha", "Rahul"], "age": [21, 22]})
df = df.set_index("roll")
print(df.index)
```

Example: Reset index back to a column

```
df = pd.DataFrame({"roll":[101,102], "name":["Asha","Rahul"]}).set_index("roll")
df = df.reset_index()
print(df.index)
```

Example: Name the index

```
df = pd.DataFrame({"name":["Asha","Rahul"]}, index=[101,102])
df.index.name = "roll"
print(df.index.name)
```

Example: Datetime index

```
dates = pd.to_datetime(["2025-08-01","2025-08-02","2025-08-03"])
df = pd.DataFrame({"present":[True, True, False]}, index=dates)
print(df.index)
```

Example: MultiIndex (two-level row index)

```
df = pd.DataFrame({
    "city":["Agra","Agra","Delhi","Delhi"],
    "name":["Asha","Rahul","Asha","Rahul"],
    "score":[85,90,92,88]
})
df = df.set_index(["city","name"])
print(df.index)
```

Example: Reindex to a new label set (adds missing rows)

```
df = pd.DataFrame({"name":["Asha","Rahul"]}, index=[101,102])
df = df.reindex([100,101,102,103])
print(df.index)
print(df.loc[100:103])
```

Example: Sort by index

```
df = pd.DataFrame({"score":[90,85,88]}, index=[103,101,102])
df = df.sort_index()
print(df.index)
```

Example: Check index uniqueness / membership

```
df = pd.DataFrame({"name":["Asha","Rahul"]}, index=[101,101])
print(df.index.is_unique)
print(101 in df.index)
```

Example: Set a custom non-numeric index

```
df = pd.DataFrame({"age":[21,22]}, index=["Asha","Rahul"])
print(df.index)                                # Index(['Asha','Rahul'], dtype='object')
print(df.loc["Asha"])                          # Row by label
```

.shape — Size of the DataFrame

- A plain tuple (rows, cols) describing the size.
- Very useful for quick checks after filters/joins/reshapes.

Example: Basic shape

```
df = pd.DataFrame({"a":[1,2,3], "b":[4,5,6]})
print(df.shape)
```

Example: After adding a column

```
df = pd.DataFrame({"a":[1,2,3]})
df["b"] = [4,5,6]
print(df.shape)
```

Example: After filtering rows

```
df = pd.DataFrame({"score":[85,90,70,95]})
df2 = df[df["score"] >= 85]
print(df.shape, df2.shape)
```

Example: After dropping duplicates

```
df = pd.DataFrame({"x":[1,1,2,2,3], "y":[10,10,20,20,30]})
print(df.drop_duplicates().shape)
```

Example: After groupby + agg

```
df = pd.DataFrame({"city":["Agra","Agra","Delhi"], "marks":[85,90,92]})
g = df.groupby("city")["marks"].mean().reset_index()
print(g.shape)
```

Example: After melt (rows increase, cols change)

```
df = pd.DataFrame({"name":["Asha","Rahul"], "math":[85,78], "eng":[90,82]})
m = df.melt(id_vars="name", var_name="subject", value_name="marks")
print(m.shape)
```

Example: After pivot (shape changes)

```
df = pd.DataFrame({"name":["Asha","Asha","Rahul","Rahul"],
"subject":["math","eng","math","eng"], "marks":[85,90,78,82]})
p = df.pivot(index="name", columns="subject", values="marks")
print(p.shape)
```

Example: Empty DataFrame

```
df = pd.DataFrame(columns=["a","b","c"])
print(df.shape)
```


Example: After merge/join

```
left = pd.DataFrame({"roll":[101,102], "name":["Asha","Rahul"]})
right = pd.DataFrame({"roll":[101,103], "score":[85,90]})
m = pd.merge(left, right, on="roll", how="left")
print(left.shape, right.shape, m.shape)      # (2,2) (2,2) (2,3)
```

.dtypes — Column Data Types

- Telling you the dtype of each column.
- Helps you catch numbers stored as text, parse dates, convert types, select only numeric columns, etc.

Example: View dtypes

```
df = pd.DataFrame({"roll":[101,102], "name":["Asha","Rahul"], "score":[85.0, 90.0]})
print(df.dtypes)
```

Example: Mixed column becomes object

```
df = pd.DataFrame({"x":[1, "two", 3.5]})
print(df.dtypes)
```

Example: Convert types with astype

```
df = pd.DataFrame({"roll":["101","102"], "score":["85","90"]})
df = df.astype({"roll":"int64", "score":"int64"})
print(df.dtypes)
```

Example: Parse to datetime dtype

```
df = pd.DataFrame({"date":["2025-08-01","2025-08-02"]})
df["date"] = pd.to_datetime(df["date"])
print(df.dtypes)
```

Example: Categorical dtype

```
df = pd.DataFrame({"city":["Agra","Delhi","Agra","Jaipur"]})
df["city"] = df["city"].astype("category")
print(df.dtypes)
```

Example: Boolean dtype + convert_dtypes()

```
df = pd.DataFrame({"present":[1,0,1]})
df2 = df.convert_dtypes()      # may turn to Int64/boolean depending on data
print(df2.dtypes)
```

Example: Select only numeric columns using dtypes

```
df = pd.DataFrame({"name":["Asha","Rahul"], "math":[85,78], "eng":[90,82]})
num_only = df.select_dtypes(include="number")
print(num_only.columns.tolist())      # ['math', 'eng']
```

Example: Per-column conversion dict

```
df = pd.DataFrame({"roll":[101,102], "score":[85.0,90.0]})  
df = df.astype({"roll":"string", "score":"int32"})  
print(df.dtypes)                                # roll string, score int32
```

Example: Detect wrong dtype (numbers-as-text) and fix

```
df = pd.DataFrame({"price":["1999","999","2499"]})  
print(df.dtypes)                                # object  
df["price"] = pd.to_numeric(df["price"])  
print(df.dtypes)                                # int64
```

Viewing DataFrame Data

Method	What it does
<code>.head(n)</code>	First n rows (default 5). Quick glance at the top of the table.
<code>.tail(n)</code>	Last n rows (default 5). Useful for recent data or end of file.
<code>.info()</code>	Structural summary: rows, columns, dtypes, non-null counts, memory usage.
<code>.describe()</code>	Statistical summary (count, mean, std, min, max, quartiles).
<code>.sample(n)</code>	Random row(s). Good for random spot-checks.

.head() — View First Rows

- Default shows first 5 rows.
- You can pass a number n to see more/less rows.
- Useful for quickly inspecting the structure and first records.

Example

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Roll": range(101, 111),
    "Name":
    ["Asha", "Rahul", "Karan", "Neha", "Anil", "Pooja", "Arjun", "Sneha", "Rohit", "Meera"],
    "Score": np.random.randint(70, 100, 10)
})
```

Example: Basic usage

```
print(df.head())
```

Example: First 3 rows only

```
print(df.head(3))
```

Example: View top 8 rows

```
print(df.head(8))
```

Example: Check structure with `.columns` + `.head()`

```
print(df.columns)
print(df.head())
```

Example: Head after sorting by Score

```
print(df.sort_values("Score", ascending=False).head())
```

Example: Head after filtering (Score > 85)

```
print(df[df["Score"] > 85].head())
```

Example: Head of only Name and Score columns

```
print(df[["Name","Score"]].head())
```

Example: Head with reset index

```
print(df.head().reset_index())
```

Example: Save top 5 rows to another DataFrame

```
top5 = df.head()
print(top5)
```

.tail() — View Last Rows

- Default shows last 5 rows.
- You can pass a number n to see last n rows.
- Good for checking data at the bottom (like totals, last date, newest records).

Example: Basic usage

```
print(df.tail())
```

Example: Last 3 rows

```
print(df.tail(3))
```

Example: Last 8 rows

```
print(df.tail(8))
```

Example: Tail after sorting by Roll

```
print(df.sort_values("Roll", ascending=True).tail())
```

Example: Tail after filtering (Score < 80)

```
print(df[df["Score"] < 80].tail())
```

Example: Tail with only Roll and Name

```
print(df[["Roll","Name"]].tail())
```

Example: Save last 5 rows to new DataFrame

```
last5 = df.tail()
print(last5)
```

Example: Tail with reset index

```
print(df.tail().reset_index())
```

.info() — DataFrame Structure Info

- Shows:
 - Column names
 - Non-null counts
 - Data types (int64, object, float64, etc.)
 - Memory usage
- Great for data cleaning — to check missing values and types.

Example: Basic usage

```
df.info()
```

Example: Info with memory usage

```
df.info(memory_usage="deep")
```

Example: Info with null values present

```
df2 = df.copy()
df2.loc[2, "Score"] = None
df2.info()
```

Example: Info of large DataFrame (1000 rows)

```
big = pd.DataFrame({"a":range(1000), "b":["x"]*1000})
big.info()
```

Example: Info showing string type

```
df2 = pd.DataFrame({"Name":["Asha", "Rahul"], "City":["Agra", "Delhi"]})
df2.info()
```

Example: Info with datetime

```
df2 = pd.DataFrame({"Date": pd.to_datetime(["2025-08-01", "2025-08-02"])})
df2.info()
```

Example: Info with categorical dtype

```
df2 = pd.DataFrame({"City":["Agra", "Delhi", "Agra"]})
df2["City"] = df2["City"].astype("category")
df2.info()
```

Example: Info of only certain columns

```
df[["Name", "Score"]].info()
```

Example: Use info after type conversion

```
df2 = df.astype({"Score":"float64"})
df2.info()
```

.describe() — Statistical Summary

- Default: Only numeric columns → count, mean, std, min, 25%, 50% (median), 75%, max.
- With include="all" → also summarizes object/categorical columns.
- Helps to quickly check distribution and range of values.

Example: Basic usage

```
print(df.describe())
```

Example: Describe only specific column

```
print(df["Score"].describe())
```

Example: Describe including object columns

```
print(df.describe(include="all"))
```

Example: Describe only object columns

```
print(df.describe(include=[object]))
```

Example: Describe categorical column

```
df2 = pd.DataFrame({"City":["Agra","Delhi","Agra","Lucknow"]})  
df2["City"] = df2["City"].astype("category")  
print(df2.describe(include="category"))
```

Example: Describe boolean column

```
df2 = pd.DataFrame({"Pass":[True,False,True,True,False]})  
print(df2.describe())
```

Example: Describe after filtering

```
print(df[df["Score"] > 85].describe())
```

Example: Describe with percentiles argument

```
print(df["Score"].describe(percentiles=[.1,.5,.9]))
```

Example: Describe transposed (rows ↔ columns)

```
print(df.describe().T)
```

.sample() — Random Sampling

- Returns random row(s) from DataFrame.
- Useful for spot-checking or reducing large datasets.
- You can choose number of samples `n`, or fraction `frac`.
- Supports `random_state` for reproducibility.

Example: Get 1 random row

```
print(df.sample())
```

Example: Get 3 random rows

```
print(df.sample(3))
```

Example: Sample fraction (20% of rows)

```
print(df.sample(frac=0.2))
```

Example: Sample with fixed seed (reproducible)

```
print(df.sample(3, random_state=42))
```

Example: Sample with replacement (rows can repeat)

```
print(df.sample(5, replace=True))
```

Example: Sample only specific columns then `.sample()` rows

```
print(df[["Name", "Score"]].sample(2))
```

Example: Sample on a filtered DataFrame

```
print(df[df["Score"]>85].sample(1))
```

Example: Sample a large dataset down to 10 rows

```
big = pd.DataFrame({"num":range(1000)})  
print(big.sample(10))
```

Example: Sample and reset index

```
print(df.sample(3).reset_index(drop=True))
```

Selecting columns (df['col'], df[['col1','col2']])

- df['col'] gives a Series (1D). Useful for vector ops, plotting one column, dtype checks, etc.
- df[['col1','col2']] gives a DataFrame (2D). Use this when you need multiple columns or want to keep a DataFrame shape.
- Column order in [['col1','col2']] is preserved (you can also reorder columns this way).
- Selecting a non-existent column raises KeyError. For dynamic code, validate your list first.
- Columns with spaces/special chars or duplicate names are still selectable with bracket syntax.
- For assignment, prefer df.loc[:, 'col'] = ... to avoid chained-assignment pitfalls—selection here focuses on reading.

DataFrame Sample

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    "Roll": range(101, 107),
    "Name": ["Asha", "Rahul", "Karan", "Neha", "Anil", "Pooja"],
    "Age": [21, 22, 20, 21, 23, 22],
    "City": ["Agra", "Delhi", "Jaipur", "Lucknow", "Kanpur", "Noida"],
    "Math": [85, 78, 92, 88, 80, 91],
    "Eng": [90, 82, 88, 84, 79, 93],
    "Sci": [88, 91, 85, 89, 87, 95]
})
```

Example: Select a single column as a Series

```
s = df['Name']
print(type(s), s.head())
```

<class 'pandas.core.series.Series'>

One column → Series (shape (n,)).

Example: Select a single column but keep DataFrame shape

```
one_col_df = df[['Name']]
print(type(one_col_df), one_col_df.shape)
```

<class 'pandas.core.frame.DataFrame'> (6, 1)

Wrap the column name in a list to keep a DataFrame.

Example: Select multiple columns (and reorder them)

```
subset = df[['Name', 'City', 'Age']]
print(subset.head())
```

Column order is exactly as listed.

Example: Select using a variable list (dynamic selection)

```
cols = ['Roll', 'Math', 'Eng']
marks = df[cols]
print(marks.head())
```

Great for functions where column names come from user/config.

Example: Compute with selected numeric columns

```
df['Average'] = df[['Math', 'Eng', 'Sci']].mean(axis=1)
print(df[['Name', 'Average']])
```

Select → compute → (optionally) assign back.

Example: Columns with spaces or “odd” names

```
df2 = df.rename(columns={'Name': 'Student Name'})

view = df2[['Student Name', 'City']]
print(view.head())
```

Brackets work fine even when names have spaces.

Example: Duplicate column names behavior

```
df_dup = pd.DataFrame([[85, 90, "Agra"], [78, 82, "Delhi"]],
                      columns=['score', 'score', 'city'])

two_scores = df_dup['score']                                     # <- DataFrame (because name is duplicated)
print(type(two_scores), two_scores.shape)                       # <class 'pandas.core.frame.DataFrame'> (2, 2)
```

With duplicates, df['name'] returns a DataFrame, not a Series.

Example: Build the list programmatically (by dtype)

```
numeric_cols = [c for c in df.columns if df[c].dtype != 'object']

num_view = df[numeric_cols]
print(numeric_cols, num_view.head())
```

Powerful pattern: discover columns → select with df[cols].

Example: Combine row filtering with column selection

```
high_scorers = df[df['Math'] >= 90][['Name', 'Math', 'Eng']]
print(high_scorers)
```

First filter rows, then pick only the columns you want.

Example: MultiIndex (hierarchical) column selection

```
df_multi = pd.DataFrame(  
    np.array([[85, 90, 78, 82],  
             [92, 88, 80, 84]]),  
    columns=pd.MultiIndex.from_tuples([  
        ('Marks', 'Math'), ('Marks', 'Eng'),  
        ('Scores', 'Math'), ('Scores', 'Eng')  
    ])  
)  
# single column (tuple key) → Series  
col_series = df_multi[('Marks', 'Math')]  
# multiple columns (list of tuples) → DataFrame  
multi_subset = df_multi[[('Marks', 'Math'), ('Scores', 'Eng')]]  
print(type(col_series), type(multi_subset))
```

Use tuples as keys for MultiIndex columns.

Important Points

- If you need to rename while selecting? do
`df[['A', 'B']].rename(columns={'A':'a', 'B':'b'})`.
- If you need to ensure columns exist? do
set containing the column names you wish to select & "XYZ" might not actually exist.
`columnSet = {'Name', 'City', 'XYZ'}`
iterates over every element in want and keeps only those that exist in df.columns.
`columnElement = [c for c in columnSet if c in df.columns]`
safely select only the columns that exist in the DataFrame.
`safe = df[columnElement]`

Final Code

```
import pandas as pd  
df = pd.DataFrame({  
    "Roll": [101, 102, 103],  
    "Name": ["Asha", "Rahul", "Karan"],  
    "City": ["Agra", "Delhi", "Jaipur"],  
    "Score": [85, 90, 88]  
})  
  
columnSet = {'Name', 'City', 'XYZ'}  
columnElement = [c for c in columnSet if c in df.columns]  
safe = df[columnElement]  
print("Columns:", columnElement)  
print(safe)
```

Selecting rows (.loc[], .iloc[], .at[], .iat[])

- `.loc[row_selector, col_selector]` → label-based (uses index/column NAMES).
 - Slices by labels are inclusive of the end (101:104 includes 104).
 - Accepts single label, list of labels, slices, boolean masks, callables.
- `.iloc[row_selector, col_selector]` → position-based (uses 0...N-1 positions).
 - Slices are exclusive of the end (0:3 gives rows at positions 0,1,2).
- `.at[row_label, col_label]` → fast scalar label lookup (get/set one cell).
- `.iat[i, j]` → fast scalar position lookup (get/set one cell).
- Prefer `.loc/.iloc` for selecting blocks; prefer `.at/.iat` for single-cell speed.

Example: Base DataFrame

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Roll": [101,102,103,104,105,106,107,108],
    "Name": ["Asha","Rahul","Karan","Neha","Anil","Pooja","Arjun","Sneha"],
    "City": ["Agra","Delhi","Jaipur","Lucknow","Kanpur","Noida","Varanasi","Gurgaon"],
    "Age": [21,22,20,21,23,22,24,21],
    "Math": [85,78,92,88,80,91,76,95],
    "Eng": [90,82,88,84,79,93,81,94],
    "Sci": [88,91,85,89,87,95,80,96],
    "Date": pd.to_datetime([
        "2025-08-01","2025-08-01","2025-08-01","2025-08-02",
        "2025-08-02","2025-08-02","2025-08-03","2025-08-03"
    ])
}).set_index("Roll")
```

.loc[] — label-based selection

Example: Single row by label

```
row_101 = df.loc[101]
print(row_101)
```

Example: Multiple rows by label list

```
rows = df.loc[[101, 104, 108]]
print(rows)
```

Example: Label slice (end is inclusive)

```
block = df.loc[101:104]
print(block)
```

includes 104

Example: Rows by boolean mask

```
print(df.loc[df["Age"] >= 22])
```

Example: Compound condition with & |

```
print(df.loc[(df["Math"] >= 90) & (df["City"] == "Delhi")])
```

Example: Rows by labels + choose specific columns

```
print(df.loc[[101,103,108], ["Name","Math","Eng"]])
```

Example: Row label slice + column label slice

```
print(df.loc[102:105, "City":"Eng"])
```

Example: Set values by condition (assignment)

```
df_loc = df.copy()
df_loc.loc[df_loc["Math"] < 80, "Math"] = 80
print(df_loc[["Name","Math"]])
```

Example: Callable selector (row-wise)

```
print(df.loc[lambda d: d["Sci"] > 90, ["Name","Sci"]])
```

Example: Membership with .isin()

```
rolls = [101, 105, 999]
print(df.loc[df.index.isin(rolls), ["Name","City"]])
```

.iloc[] — position-based selection (end of a slice is exclusive)

Example: Single row by position

```
first_row = df.iloc[0] # row at position 0 (Roll 101)
print(first_row)
```

Example: Multiple rows by positions

```
rows = df.iloc[[0,3,5]]
print(rows)
```

Example: Slice first three rows (0,1,2)

```
top3 = df.iloc[0:3]
print(top3)
```

Example: Row & column positions together

```
# rows 0..2, columns 0..2 (index col is not in df; first col is Name at position 0)
print(df.iloc[0:3, 0:3])
```

Example: Negative indices

```
last_row = df.iloc[-1] # last row (Roll 108)
print(last_row)
```

Example: Step slice (every second row)

```
print(df.iloc[::2])
```

Example: Reorder rows by positions

```
print(df.iloc[[2,0,1]])
```

Example: Pick rows by positions + named columns (mix .iloc + col list via .loc-like)

```
math_eng_first4 = df.iloc[0:4][["Math","Eng"]]          # (two-step)
print(math_eng_first4)
# or purely position-based:
math_eng_first4_pos = df.iloc[0:4, [df.columns.get_loc("Math"),
df.columns.get_loc("Eng")]]
print(math_eng_first4_pos)
```

Example: 2D slice: middle block

```
mid = df.iloc[2:6, 2:5]                                # rows pos 2..5, cols pos 2..4
print(mid)
```

.at[] — fast scalar by labels

Use exact row index label + exact column label for one cell.

Example: Get a single cell (label, label)

```
print(df.at[101, "Math"])                               # 85
```

Example: Set a single cell

```
df.at[101, "Math"] = 86
print(df.at[101, "Math"])
```

Example: Use with datetime index

```
df_dates = df.reset_index().set_index("Date")
data = df_dates.at[pd.Timestamp("2025-08-01"), "Name"]
print(data)
```

Example: Create a new column via .at (others become NaN)

```
df.at[101, "Section"] = "A"                             # creates 'Section' col; only row 101 set
print(df)
```

Example: Difference from .loc (scalar vs Series)

```
scalar = df.at[104, "Eng"]                               # returns a Python scalar
series = df.loc[104, "Eng":"Eng"]                       # returns a one-element Series
print(scalar)
print(series)
```

Example: Speed pattern in loops (illustrative)

```
for r in [101,102,103]:
    df.at[r, "Sci"] += 1
print(df)
```

Example: Handle missing label safely

```
r = 999
val = df.at[r, "Math"] if r in df.index else None
print(val)
```

Example: Set None / NaN explicitly

```
df.at[105, "City"] = None
print(df)
```

Example: Row label & column label from variables

```
r, c = 106, "Eng"
df.at[r, c] = df.at[r, c] + 2
print(df)
```

.iat[] — fast scalar by positions (10 examples)

Use row i, column j (0-based positions) for one cell.

Example: Get a single cell by position

```
v = df.iat[0, df.columns.get_loc("Math")]
print(v) # first row's Math
```

Example: Set a single cell by position

```
df.iat[0, df.columns.get_loc("Math")] = 90
print(df)
```

Example: Negative positions work

```
last_city = df.iat[-1, df.columns.get_loc("City")]
print(last_city)
```

Example: Out-of-bounds raises IndexError

```
df.iat[999, 0] # IndexError
```

Example: Speed pattern inside loop

```
math_col = df.columns.get_loc("Math")
for i in range(len(df)):
    df.iat[i, math_col] += 1
print(df)
```

Example: Compare .iat vs .iloc (scalar)

```
i = 2; j = df.columns.get_loc("Eng")
a = df.iat[i, j] # scalar
b = df.iloc[i, j] # also scalar; .iat is faster for scalar ops
print(a)
print(b)
```

Example: Use with computed positions

```
row_pos = df.index.get_indexer_for([104])[0] # find position of Roll 104
col_pos = df.columns.get_loc("Sci")
df.iat[row_pos, col_pos] = 99
print(df)
```

Filtering with multiple conditions?

- When you want to filter data based on more than one condition, you use & (AND), | (OR) & ~ (NOT)
- Always wrap conditions in parentheses ()
- Use bitwise operators: &, |, ~ (not and, or, not)
- Apply conditions on columns using boolean logic
- Combine with .loc[] or directly filter the DataFrame

Example: Sample DataFrame

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Arjun', 'Bhavna', 'Chetan', 'Divya', 'Farhan', 'Geeta'],
    'Age': [23, 29, 35, 28, 22, 40],
    'Gender': ['M', 'F', 'M', 'F', 'M', 'F'],
    'Score': [88, 76, 91, 64, 45, 83],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'HR', 'IT']
})
```

Example: Age > 25 AND Score > 80

```
print(df[(df['Age'] > 25) & (df['Score'] > 80)])
```

Example: Department is IT OR HR

```
print(df[(df['Department'] == 'IT') | (df['Department'] == 'HR')])
```

Example: NOT from HR department

```
print(df[~(df['Department'] == 'HR')])
```

Example: Male AND Score >= 85

```
print(df[(df['Gender'] == 'M') & (df['Score'] >= 85)])
```

Example: Female OR Score < 50

```
print(df[(df['Gender'] == 'F') | (df['Score'] < 50)])
```

Example: Female from IT department

```
print(df[(df['Gender'] == 'F') & (df['Department'] == 'IT')])
```

Example: Score between 70 and 90 AND Age below 30

```
print(df[(df['Score'] >= 70) & (df['Score'] <= 90) & (df['Age'] < 30)])
```

Example: Name is not 'Farhan' AND Department is not HR

```
print(df[(df['Name'] != 'Farhan') & (df['Department'] != 'HR')])
```

Example: (IT or Finance) AND Score > 70

```
print(df[((df['Department'] == 'IT') | (df['Department'] == 'Finance')) & (df['Score'] > 70)])
```

Example: NOT (Score > 80 OR Age < 25)

This uses the NOT ~ operator on the full condition:

```
print(df[~((df['Score'] > 80) | (df['Age'] < 25))])
```

Example: Chaining with .loc[]

```
print(df.loc[(df['Gender'] == 'F') & (df['Score'] > 75), ['Name', 'Score']])
```

.query() — Filtering Rows the Smart Way

.query() is a powerful method in Pandas that lets you filter rows using string-based expressions, similar to SQL-style syntax.

Syntax

```
DataFrame.query("condition")
```

Why use .query() instead of .loc[]?

.query() is better when...	Example
You want cleaner, readable filters	df.query("Age > 25 and Department == 'HR'")
You're chaining multiple filters	Instead of deeply nested .loc[]
You're using variables inside filters	df.query("Score > @cutoff")

Example: Filter students older than 30

```
print(df.query('Age > 30'))
```

Example: Filter Female candidates

```
print(df.query('Gender == "F"'))
```


Example: Get students with Score $\geq$ 80

```
print(df.query('Score >= 80'))
```

Example: IT department with score above 85

```
print(df.query('Department == "IT" and Score > 85'))
```

Example: Low score OR HR department

```
print(df.query('Score < 50 or Department == "HR"))
```

Example: Exclude Divya

```
print(df.query('Name != "Divya"))
```

Example: Filter with multiple conditions grouped using parentheses

```
print(df.query('(Department == "IT") & (Score >= 80) & (Age < 30))
```

Example: Get only non-male candidates (i.e., Female)

```
print(df.query('~(Gender == "M"))
```

Example: Filter for specific names like Amit or Score 45

```
print(df.query('Name == "Amit" or Score == 45'))
```

Example: Filter Age between 25 to 35

```
print(df.query('25 <= Age <= 35'))
```

Example: Use external Python variable in .query()

```
cutoff = 80
```

```
print(df.query('Score > @cutoff'))
```

Example: Columns with Spaces or Special Characters

```
print(df.query('`Student Name` == "Geeta"))
```

Use backticks `` around the column name.

Understanding Missing Values in Pandas

Missing values are usually represented by:

- NaN (Not a Number) for float/int objects.
- None for object data types.

Checking for Missing Values

- `.isna()` – Checks if a value is missing (returns True)
- `.notna()` – Checks if a value is not missing (returns True)

`.isna()` Examples

Example: On Series

```
import pandas as pd
import numpy as np
s = pd.Series([10, None, 30])
print(s.isna())
```

Example: Count missing

```
s = pd.Series([1, np.nan, 3, np.nan])
print(s.isna().sum())
```

Example: Mask selection

```
print(s[s.isna()])
```

Example: Check if any missing

```
print(s.isna().any())
```

Example: Replace using condition

```
s[s.isna()] = 0
print(s)
```

Example: Boolean mask for missing detection

```
mask = s.isna()
print("Missing at indexes:", s[mask].index.tolist())
```

Example: Working with strings

```
s = pd.Series(['one', None, 'three'])
print(s.isna())
```

Example: Series with datetime

```
s = pd.Series([pd.NaT, pd.Timestamp('2022-01-01')])
print(s.isna())
```

Example: Series with all non-missing

```
s = pd.Series([10, 20, 30])
print(s.isna())
```

Example: Series with all missing

```
s = pd.Series([None, np.nan, None])
print(s.isna())
```

Example: On DataFrame

```
df = pd.DataFrame({'A': [1, np.nan, 3], 'B': [4, 5, np.nan]})
print(df.isna())
```

Example: Use with sum() to count missing values

```
print(df.isna().sum())
```

Example: Check if any value is missing

```
print(df.isna().any().any())
```

Example: Row-wise missing count

```
print(df.isna().sum(axis=1))
```

Example: Apply mask using isna()

```
print(df[df['A'].isna()])
```

Example: Count missing per column

```
missing_counts = df.isna().sum()
for col in df.columns:
    print(f"{col}: {missing_counts[col]} missing")
```

Example: With datetime data

```
df = pd.DataFrame({'date': [pd.NaT, pd.Timestamp('2023-01-01'), None]})
print(df.isna())
```

Example: Detect missing after merging

```
df1 = pd.DataFrame({'id': [1, 2], 'name': ['Neha', 'Tushar']})
df2 = pd.DataFrame({'id': [1, 3]})
merged = df2.merge(df1, on='id', how='left')
print(merged.isna())
```

Example: Used with .loc for assignment

```
df.loc[df['A'].isna(), 'A'] = 0
print(df)
```

.notna() Examples

Example: On Series

```
s = pd.Series([None, 'Learn', 'Earn'])  
print(s.notna())
```

Example: Count non-missing

```
print(s.notna().sum())
```

Example: Filter valid entries

```
print(s[s.notna()])
```

Example: With assignment

```
s[s.notna()] += 1  
print(s)
```

Example: Series with strings

```
s = pd.Series(['x', None, 'z'])  
print(s.notna())
```

Example: With datetime

```
s = pd.Series([None, pd.Timestamp('2023-01-01')])  
print(s.notna())
```

Example: Loop through valid values

```
for val in s[s.notna():]  
    print(val)
```

Example: All values present

```
s = pd.Series([1, 2, 3])  
print(s.notna())
```

Example: All values missing

```
s = pd.Series([None, np.nan])  
print(s.notna())
```

Example: Apply with condition

```
s = pd.Series([1, None, 3])  
s = s.apply(lambda x: x * 2 if pd.notna(x) else 0)  
print(s)
```

Example: Filter non-null rows

```
df = pd.DataFrame({'X': [1, 2, None], 'Y': ['a', None, 'c']})  
print(df[df['X'].notna()])
```

Example: Count non-null values

```
print(df.notna().sum())
```

Example: Use in loop

```
for i in df.index:  
    if df.at[i, 'Y'] is not None:  
        print(df.at[i, 'Y'])
```

Example: Replace nulls using notna()

```
df['X'] = df['X'].where(df['X'].notna(), 0)  
print(df)
```

Example: Assign constant to non-null values

```
df['flag'] = df['Y'].notna()  
print(df)
```

Example: Use for conditional calculations

```
df['length'] = df['Y'].apply(lambda x: len(x) if pd.notna(x) else 0)  
print(df)
```

Example: With datetime

```
df = pd.DataFrame({'D': [pd.NaT, pd.Timestamp('2024-01-01')]})  
print(df[df['D'].notna()])
```

Example: Custom function using notna

```
def clean(x):  
    return x if pd.notna(x) else 'Missing'  
df['Y_clean'] = df['Y'].apply(clean)  
print(df)
```

Example: Boolean mask with notna()

```
mask = df['X'].notna()  
print(df[mask])
```

Dropping Missing Values

.dropna() – Removes rows or columns containing missing values.

Example: Drop single NaN

```
s = pd.Series([1, np.nan, 3])  
print(s.dropna())
```

Example: Series with all NaN

```
s = pd.Series([None, np.nan])  
print(s.dropna())
```

Example: Series with all valid

```
s = pd.Series([10, 20])  
print(s.dropna())
```

Example: Drop inplace

```
s = pd.Series([1, None, 3])  
s.dropna(inplace=True)  
print(s)
```

Example: Reset index after dropping

```
s = pd.Series([1, None, 2])  
clean_s = s.dropna().reset_index(drop=True)  
print(clean_s)
```

Example: Drop from string series

```
s = pd.Series(['apple', None, 'banana'])  
print(s.dropna())
```

Example: Drop datetime NaT

```
s = pd.Series([pd.NaT, pd.Timestamp('2023-01-01')])  
print(s.dropna())
```

Example: Drop with duplicated NaNs

```
s = pd.Series([1, np.nan, np.nan, 2])  
print(s.dropna())
```

Example: Drop but check before

```
if s.isna().any():  
    print(s.dropna())
```

Example: Length difference before and after

```
s = pd.Series([10, None, 30])
print("Before:", len(s))
print("After :", len(s.dropna()))
```

Example: Drop rows with any missing values

```
df = pd.DataFrame({
    'A': [1, None, 3],
    'B': [None, 5, None],
    'C': [7, 8, 9]
})
print(df.dropna())
```

Example: Drop columns with any missing values

```
print(df.dropna(axis=1))
```

Example: Drop rows where all values are NaN

```
df2 = pd.DataFrame({'A': [None, None], 'B': [None, 2]})
print(df2.dropna(how='all'))
```

Example: Keep rows with at least 2 non-null values

```
print(df.dropna(thresh=2))
```

Example: Drop rows only if 'A' is NaN

```
print(df.dropna(subset=['A']))
```

Example: Drop rows only if 'B' or 'C' is NaN

```
print(df.dropna(subset=['B', 'C']))
```

Example: Drop in place

```
df_copy = df.copy()
df_copy.dropna(inplace=True)
print(df_copy)
```

Example: Reset index after drop

```
new_df = df.dropna().reset_index(drop=True)
print(new_df)
```

Example: With datetime columns

```
df_dt = pd.DataFrame({'date': [None, pd.Timestamp('2025-01-01')]})
print(df_dt.dropna())
```

Example: Chain dropna with other methods

```
print(df.dropna().sort_values(by='C'))
```

Filling Missing Values

.fillna() – Fills missing data with a value or method.

Example: Fill with constant

```
s = pd.Series([1, None, 3])  
print(s.fillna(0))
```

Example: Fill with string

```
s = pd.Series(['a', None, 'c'])  
print(s.fillna("missing"))
```

Example: Fill with mean

```
s = pd.Series([1, None, 3])  
print(s.fillna(s.mean()))
```

Example: Fill with median

```
s = pd.Series([1, None, 7])  
print(s.fillna(s.median()))
```

Example: Fill with mode

```
s = pd.Series([None, 'x', 'x', None])  
print(s.fillna(s.mode()[0]))
```

Example: Forward fill

```
s = pd.Series([None, 1, None, 2])  
print(s.fillna(method='ffill'))
```

Example: Backward fill

```
s = pd.Series([None, 1, None, 2])  
print(s.fillna(method='bfill'))
```

Example: Fill with conditional value

```
s = pd.Series([1, None, 3])  
s[s.isna()] = 100  
  
print(s)
```

Example: Inplace filling

```
s = pd.Series([None, 2])  
s.fillna(0, inplace=True)  
  
print(s)
```


Example: With datetime values

```
s = pd.Series([pd.NaT, pd.Timestamp('2024-01-01')])
print(s.fillna(method='bfill'))
```

Example: Fill with constant (numeric)

```
df = pd.DataFrame({
    'A': [None, 2, None, 4],
    'B': ['x', None, 'z', None]
})
```

```
print(df.fillna(0))
```

Example: Fill with constant (string)

```
print(df.fillna("Missing"))
```

Example: Fill column A with mean

```
df['A'] = df['A'].fillna(df['A'].mean())
print(df)
```

Example: Fill column A with median

```
df['A'] = df['A'].fillna(df['A'].median())
print(df)
```

Example: Fill column B with mode

```
df['B'] = df['B'].fillna(df['B'].mode()[0])
print(df)
```

Example: Use method='ffill' (forward fill)

```
print(df.fillna(method='ffill'))
```

Example: Use method='bfill' (backward fill)

```
print(df.fillna(method='bfill'))
```

Example: Fill with dictionary

```
print(df.fillna({'A': 100, 'B': 'Unknown'}))
```

Example: Fill only specific column

```
df['A'] = df['A'].fillna(999)
print(df)
```

Example: Fill inplace

```
df.fillna('N/A', inplace=True)
print(df)
```

Detecting & Handling Duplicates in Pandas

Duplicates can occur when the same row or value appears multiple times, either accidentally or due to data merging, scraping, etc.

Common Methods:

- `.duplicated()` – Detects duplicate rows or values.
- `.drop_duplicates()` – Removes duplicate rows or values.

`.duplicated()` – Detecting duplicates

Example: Simple series with repeated values

```
s = pd.Series([1, 2, 2, 3, 1])  
print(s.duplicated())
```

Example: Show only duplicate values

```
print(s[s.duplicated()])
```

Example: Count duplicates

```
print(s.duplicated().sum())
```

Example: Get unique duplicates

```
print(s[s.duplicated()].unique())
```

Example: Keep last instead of first

```
print(s.duplicated(keep='last'))
```

Example: Mark all duplicates as True

```
print(s.duplicated(keep=False))
```

Example: Detect duplicates in a string Series

```
s = pd.Series(['apple', 'banana', 'apple', 'cherry'])  
print(s.duplicated())
```

Example: With NaN values

```
s = pd.Series([1, np.nan, np.nan, 2, 1])  
print(s.duplicated())
```

Example: Find index of duplicates

```
duplicates = s[s.duplicated()]  
print(duplicates.index.tolist())
```

Example: Custom deduplication logic

```
# Drop duplicates only above value > 1
s = pd.Series([1, 2, 2, 3, 1])
print(s[(s > 1) & s.duplicated()])
```

Example: Detect duplicate rows

```
df = pd.DataFrame({'A': [1, 2, 2, 3], 'B': ['x', 'y', 'y', 'z']})
print(df.duplicated())
```

Example: Detect duplicates based on a single column

```
print(df.duplicated(subset='A'))
```

Example: Keep last occurrence

```
print(df.duplicated(keep='last'))
```

Example: Keep all duplicates

```
print(df.duplicated(keep=False))
```

Example: Filter only duplicate rows

```
print(df[df.duplicated()])
```

Example: Find duplicates in specific column

```
print(df[df.duplicated(subset='B')])
```

Example: Multiple columns as subset

```
df = pd.DataFrame({'A': [1, 2, 2, 1], 'B': ['x', 'y', 'y', 'x']})
print(df.duplicated(subset=['A', 'B']))
```

Example: With NaNs in data

```
df = pd.DataFrame({'A': [1, 2, np.nan, 1], 'B': [np.nan, 2, 3, np.nan]})
print(df.duplicated())
```

Example: Count total duplicates

```
print("Total duplicates:", df.duplicated().sum())
```

Example: Mark and filter both duplicates and non-duplicates

```
df['is_duplicate'] = df.duplicated()
print(df)
```

.drop_duplicates() – Removing duplicates

Example: Drop duplicate values

```
s = pd.Series([1, 2, 2, 3, 1])  
print(s.drop_duplicates())
```

Example: Keep last

```
print(s.drop_duplicates(keep='last'))
```

Example: Drop all duplicates (keep=False)

```
print(s.drop_duplicates(keep=False))
```

Example: With strings

```
s = pd.Series(['a', 'b', 'a', 'c'])  
print(s.drop_duplicates())
```

Example: With NaN values

```
s = pd.Series([1, np.nan, np.nan, 2, 1])  
print(s.drop_duplicates())
```

Example: Count after removing duplicates

```
print(len(s.drop_duplicates()))
```

Example: Reset index after dropping duplicates

```
print(s.drop_duplicates().reset_index(drop=True))
```

Example: Drop duplicates in-place

```
s2 = s.copy()  
s2.drop_duplicates(inplace=True)  
  
print(s2)
```

Example: Drop duplicates and sort

```
print(s.drop_duplicates().sort_values())
```

Example: Filter unique duplicate values only

```
unique_dupes = s[s.duplicated(keep=False)].drop_duplicates()  
print(unique_dupes)
```

Example: Drop exact duplicate rows

```
df = pd.DataFrame({'A': [1, 2, 2, 3], 'B': ['x', 'y', 'y', 'z']})  
print(df.drop_duplicates())
```

Example: Drop duplicates based on one column

```
print(df.drop_duplicates(subset='A'))
```

Example: Drop and keep last

```
print(df.drop_duplicates(keep='last'))
```

Example: Drop keeping none (remove all repeating)

```
print(df.drop_duplicates(keep=False))
```

Example: Inplace drop

```
df2 = df.copy()
df2.drop_duplicates(inplace=True)
print(df2)
```

Example: With NaN values

```
df = pd.DataFrame({'A': [1, np.nan, 1, np.nan]})
print(df.drop_duplicates())
```

Example: Drop based on multiple columns

```
df = pd.DataFrame({'A': [1, 2, 1], 'B': ['x', 'y', 'x']})
print(df.drop_duplicates(subset=['A', 'B']))
```

Example: Count after drop

```
print("After drop:", len(df.drop_duplicates()))
```

Example: Reset index

```
df_clean = df.drop_duplicates().reset_index(drop=True)
print(df_clean)
```

Example: Filter then drop

```
filtered = df[df['A'].notna()]
print(filtered.drop_duplicates())
```

Renaming Columns and Indexes in Pandas

In Pandas, we use:

- `.rename()` to rename specific columns or indexes.
- `.set_axis()` to rename all at once.
- `.columns` and `.index` to directly assign new names.

Syntax

```
df.rename(columns={'old_name': 'new_name'}, index={old_index: new_index})
```

A Series doesn't have columns, only an index and values. So we rename the index, not columns.

Example: Rename one index label

```
import pandas as pd
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s.rename(index={'a': 'alpha'}))
```

Example: Rename multiple index labels

```
print(s.rename(index={'a': 'alpha', 'b': 'beta'}))
```

Example: Rename using lambda

```
print(s.rename(lambda x: x.upper()))
```

Example: Rename to numeric index

```
print(s.rename(index={'a': 1, 'b': 2, 'c': 3}))
```

Example: Change index completely

```
s.index = ['x', 'y', 'z']
print(s)
```

Example: Rename using `.set_axis()` (rare for Series)

```
s2 = pd.Series([100, 200, 300])
s2 = s2.set_axis(['first', 'second', 'third'])
print(s2)
```

Example: Reset index (convert index to column)

```
print(s.reset_index())
```

Example: Reset and rename Series name

```
s.name = 'Marks'
print(s)
```

Example: Reindex with new order

```
print(s.reindex(['z', 'y', 'x']))
```

Example: Rename then access value

```
s2 = s.rename(index={'x': 'X'})
print(s2['X'])
```

Example: Rename one column of DataFrame

```
df = pd.DataFrame({
    'A': [1, 2],
    'B': [3, 4]
}, index=['row1', 'row2'])
print(df.rename(columns={'A': 'Alpha'}))
```

Example: Rename multiple columns

```
print(df.rename(columns={'A': 'Alpha', 'B': 'Beta'}))
```

Example: Rename index labels

```
print(df.rename(index={'row1': 'r1'}))
```

Example: Use str.upper() for all column names

```
df.columns = df.columns.str.upper()
print(df)
```

Example: Rename using set_axis() for columns

```
print(df.set_axis(['X', 'Y'], axis=1))
```

Example: Rename using set_axis() for index

```
df2 = df.set_axis(['first', 'second'], axis=0)
print(df2)
```

Example: Rename with inplace=True

```
df_copy = df.copy()
df_copy.rename(columns={'A': 'Col1'}, inplace=True)
print(df_copy)
```

Example: Prefix/Suffix all column names

```
df.columns = ['col_' + col for col in df.columns]
print(df)
```

Example: Reset and rename index

```
df_reset = df.reset_index().rename(columns={'index': 'RowLabel'})
print(df_reset)
```

Example: Rename using lambda

```
df.columns = df.columns.map(lambda x: x.lower())
print(df)
```

Changing Data Types with .astype()

Pandas provides .astype() to convert data types from int to float, object (string), bool, category, etc.

Example: int → float

```
s = pd.Series([1, 2, 3])
print(s.astype(float))
```

Example: int → str

```
print(s.astype(str))
```

Example: str → int

```
s = pd.Series(['1', '2', '3'])
print(s.astype(int))
```

Example: str → float

```
print(s.astype(float))
```

Example: int → bool

```
s = pd.Series([0, 1, 2])
print(s.astype(bool))
```

Example: int → category

```
s = pd.Series([1, 2, 2, 3])
print(s.astype('category'))
```

Example: float32 conversion

```
s = pd.Series([1, 2, 3])
print(s.astype(np.float32))
```

Example: Using .astype(object)

```
s = pd.Series([1, 2])
print(s.astype(object))
```

Example: Handling error with errors='ignore'

```
s = pd.Series(['10', 'x', '20'])
print(s.astype(int, errors='ignore'))
```

Example: str → datetime

```
s = pd.Series(['2022-01-01', '2022-02-01'])
print(pd.to_datetime(s))
```


Example: Convert int to float

```
df = pd.DataFrame({
    'int_col': [1, 2, 3],
    'float_col': [1.1, 2.2, 3.3],
    'str_col': ['4', '5', '6']
})
print(df['int_col'].astype(float))
```

Example: Convert float to int

```
print(df['float_col'].astype(int))
```

Example: Convert string to integer

```
print(df['str_col'].astype(int))
```

Example: Convert string to float

```
print(df['str_col'].astype(float))
```

Example: Convert column to string

```
print(df['int_col'].astype(str))
```

Example: Convert to boolean

```
print(df['int_col'].astype(bool))
```

Example: Convert to category

```
print(df['int_col'].astype('category'))
```

Example: Convert multiple columns using astype()

```
df2 = df.astype({'int_col': float, 'str_col': int})
print(df2)
```

Example: Convert using numpy type

```
print(df['int_col'].astype(np.float32))
```

Example: Handling conversion error using errors param

```
s = pd.Series(['1', '2', 'x'])
print(s.astype(int, errors='ignore'))
```

String Operations using .str accessor

The .str accessor allows you to apply vectorized string functions on Series of strings (not on entire DataFrame).

Example: Convert to uppercase

```
data = pd.Series(['apple', 'banana', 'cherry', 'apple pie', 'banana split'])
print(data.str.upper())
```

Example: Convert to lowercase

```
print(data.str.lower())
```

Example: Capitalize each string

```
print(data.str.capitalize())
```

Example: Check if string contains substring

```
print(data.str.contains('apple'))
```

Example: Replace substring

```
print(data.str.replace('apple', 'mango'))
```

Example: Extract characters using regex

```
print(data.str.extract(r'(\w+)$'))          # last word
```

Example: Count character occurrences

```
print(data.str.count('a'))
```

Example: String startswith()

```
print(data.str.startswith('ba'))
```

Example: String endswith()

```
print(data.str.endswith('e'))
```

Example: Padding and alignment

```
print(data.str.pad(width=15, side='right', fillchar='-'))
```

String Operations with .str on DataFrames

.str can only be applied to Series of string type, so when working with a DataFrame, you must apply .str to each column individually (not the entire DataFrame).

DataFrame Sample

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Aarav Sharma', 'Meera Iyer', 'Rajiv Kumar', 'Tanisha Singh', 'Vikram Mehta'],
    'Email': ['aarav@rediffmail.com', 'meera@gmail.com', 'rajiv@yahoo.co.in',
'tanisha@outlook.com', 'vikram@gmail.com']
})
```

Example: Convert names to uppercase

```
df['Name_Upper'] = df['Name'].str.upper()
print(df[['Name', 'Name_Upper']])
```

Example: Convert emails to lowercase

```
df['Email_Lower'] = df['Email'].str.lower()
print(df[['Email', 'Email_Lower']])
```

Example: Check if name starts with 'M'

```
df['StartsWith_M'] = df['Name'].str.startswith('M')
print(df[['Name', 'StartsWith_M']])
```

Example: Check if email contains 'gmail'

```
df['Is_Gmail'] = df['Email'].str.contains('gmail')
print(df[['Email', 'Is_Gmail']])
```

Example: Extract domain from email

```
df['Domain'] = df['Email'].str.extract(r'@([\w\.]+)')
print(df[['Email', 'Domain']])
```

Example: Length of names

```
df['Name_Length'] = df['Name'].str.len()
print(df[['Name', 'Name_Length']])
```

Example: Replace 'gmail' with 'learn2earnlabs'

```
df['Email_Custom'] = df['Email'].str.replace('gmail', 'learn2earnlabs', regex=True)
print(df[['Email', 'Email_Custom']])
```

Example: Convert names to Title Case (first letters capital)

```
df['Name_Title'] = df['Name'].str.title()
print(df[['Name', 'Name_Title']])
```

Example: Check if email ends with '.com'

```
df['EndsWith_com'] = df['Email'].str.endswith('.com')  
print(df[['Email', 'EndsWith_com']])
```

Example: Pad names to fixed width with '*'

```
df['Name_Padded'] = df['Name'].str.pad(width=20, side='right', fillchar='*')  
print(df[['Name', 'Name_Padded']])
```

Sorting Data in Pandas

In Pandas, sorting allows us to organize data either by values (within a column) or by index labels (row or column headers). This makes it easier to perform analysis, detect patterns, and visualize information.

`.sort_values()`

- `.sort_values()` is a method used to sort data based on values in either a Series or a DataFrame.
- It is commonly used when you want to arrange your data in ascending or descending order by actual data content, such as marks, prices, names, or any other column values.
- It is applicable to:
 - **Series** – sorts based on the values.
 - **DataFrame** – sorts based on one or more columns.

Key Parameters

Parameter	Description
<code>by</code>	The column name(s) to sort by (for DataFrame only).
<code>ascending</code>	Boolean or list of booleans (default True) to sort ascending or descending.
<code>axis</code>	0 for rows (default), 1 for columns (rarely used in <code>sort_values</code>).
<code>inplace</code>	If True, modifies the original object. Default is False.
<code>na_position</code>	'first' or 'last' – position of NaNs in sorted output. Default is 'last'.

Behavior

- **Series**: sorts the values and reorders the index accordingly.
- **DataFrame**: rearranges the rows based on values in specified column(s).
- Does not affect column order (unlike `sort_index(axis=1)`).

Series Examples

Example: Sort marks

```
s = pd.Series([50, 80, 60], index=["Ravi", "Neha", "Amit"])
print(s.sort_values())
```

Example: Sort in descending order

```
print(s.sort_values(ascending=False))
```

Example: With duplicate values

```
s = pd.Series([60, 80, 60], index=["Ravi", "Neha", "Amit"])
print(s.sort_values())
```

Example: With missing values (NaN)

```
s = pd.Series([70, None, 50], index=["Amit", "Neha", "Ravi"])
print(s.sort_values(na_position="first"))
```

Example: Sorting by temperatures

```
s = pd.Series([25.5, 30.0, 28.3], index=["Monday", "Tuesday", "Wednesday"])
print(s.sort_values())
```

Example: Sorting by negative values

```
s = pd.Series([-3, 0, 2, -5])
print(s.sort_values())
```

Example: Sorting product ratings

```
s = pd.Series([4.3, 4.8, 3.5], index=["ProductA", "ProductB", "ProductC"])
print(s.sort_values(ascending=False))
```

Example: Sorting exam ranks

```
s = pd.Series([3, 1, 2], index=["Rahul", "Pooja", "Aman"])
print(s.sort_values())
```

Example: Sorting by salary

```
s = pd.Series([45000, 60000, 55000], index=["Employee1", "Employee2",
"Employee3"])
print(s.sort_values())
```

Example: Sorting with mixed index

```
s = pd.Series([2.5, 1.2, 4.8], index=[105, 101, 110])
print(s.sort_values())
```

DataFrame Examples

Example: Sort by one column

```
df = pd.DataFrame({"Name": ["Amit", "Neha", "Ravi"], "Marks": [60, 90, 70]})
print(df.sort_values(by="Marks"))
```

Example: Sort by one column (descending)

```
print(df.sort_values(by="Marks", ascending=False))
```

Example: Sort by multiple columns

```
df = pd.DataFrame({"Name": ["Amit", "Neha", "Neha"], "Marks": [60, 90, 70]})
print(df.sort_values(by=["Name", "Marks"]))
```

Example: Sort with NaN values

```
df = pd.DataFrame({"Name": ["Amit", "Neha", "Ravi"], "Marks": [60, None, 70]})
print(df.sort_values(by="Marks", na_position="first"))
```

Example: Sort by age and name

```
df = pd.DataFrame({"Name": ["Amit", "Neha", "Ravi"], "Age": [22, 20, 22]})  
print(df.sort_values(by=["Age", "Name"]))
```

Example: Sort by index then by column

```
df.index = ["b", "c", "a"]  
print(df.sort_values(by="Age").sort_index())
```

Example: Sort by string column

```
df = pd.DataFrame({"City": ["Delhi", "Agra", "Mumbai"], "Temp": [32, 35, 31]})  
print(df.sort_values(by="City"))
```

Example: Sort by numeric column with float

```
df = pd.DataFrame({"Product": ["A", "B", "C"], "Rating": [4.3, 4.9, 4.1]})  
print(df.sort_values(by="Rating"))
```

Example: Sort by date column

```
df = pd.DataFrame({"Date": pd.to_datetime(["2023-01-01", "2023-03-01", "2023-02-01"]), "Sales": [200, 300, 250]})  
print(df.sort_values(by="Date"))
```

Example: Sort by boolean column

```
df = pd.DataFrame({"Name": ["Amit", "Neha", "Ravi"], "Pass": [True, False, True]})  
print(df.sort_values(by="Pass"))
```

sort_index()

- `.sort_index()` is used to sort data by index labels rather than the values.
- It is particularly useful when your data has meaningful row or column indexes that need to be ordered alphabetically, numerically, or chronologically.
- It is also applicable to:
 - **Series** – sorts by the index (row labels).
 - **DataFrame** – can sort by row index (axis=0) or column index (axis=1).

Key Parameters

Parameter	Description
<code>axis</code>	0 (default) to sort row index, 1 to sort column names.
<code>ascending</code>	True for ascending, False for descending order.
<code>inplace</code>	True modifies the object in-place. Default is False.
<code>na_position</code>	'first' or 'last' – position of NaNs in index (if any). Default is 'last'.

Behavior

- It reorders the data based on index names or labels, not actual values in the cells.
- For hierarchical (MultiIndex) data, it sorts at each level.
- If axis=1 is set for a DataFrame, it sorts the column names instead of rows.

Series Examples

Example: Sort alphabetic index

```
s = pd.Series([10, 20, 30], index=["b", "c", "a"])
print(s.sort_index())
```

Example: Reverse sort index

```
print(s.sort_index(ascending=False))
```

Example: Sort numeric index

```
s = pd.Series([100, 200, 150], index=[3, 1, 2])
print(s.sort_index())
```

Example: Sort date index

```
s = pd.Series([100, 200, 150], index=pd.to_datetime(["2023-12-01", "2023-11-01",
"2023-12-05"]))
print(s.sort_index())
```

Example: Sort mixed-type index (strings)

```
s = pd.Series([10, 20, 30], index=["X", "Z", "Y"])
print(s.sort_index())
```

Example: Sort with custom index (e.g. product IDs)

```
s = pd.Series([90, 70, 85], index=["P003", "P001", "P002"])
print(s.sort_index())
```

Example: Sort large integer index

```
s = pd.Series([5, 2, 9], index=[1002, 1001, 1003])
print(s.sort_index())
```

Example: Sort boolean index

```
s = pd.Series([90, 70], index=[True, False])
print(s.sort_index())
```

Example: Sort with tuple index

```
s = pd.Series([100, 200, 300], index=[("Delhi", 1), ("Mumbai", 2), ("Delhi", 2)])
print(s.sort_index())
```


Example: Sort with hierarchical index

```
arrays = [["Math", "Math", "Eng"], [2023, 2022, 2023]]
index = pd.MultiIndex.from_arrays(arrays)
s = pd.Series([85, 90, 88], index=index)
print(s.sort_index())
```

DataFrame Examples

Example: Sort by row index alphabetically

```
df = pd.DataFrame({"Marks": [60, 90, 70]}, index=["c", "a", "b"])
print(df.sort_index())
```

Example: Sort by row index descending

```
print(df.sort_index(ascending=False))
```

Example: Sort columns alphabetically

```
df = pd.DataFrame({"B": [1, 2], "A": [3, 4]})
print(df.sort_index(axis=1))
```

Example: Sort by integer index

```
df = pd.DataFrame({"Value": [20, 10, 30]}, index=[3, 1, 2])
print(df.sort_index())
```

Example: Sort with multiindex (row index)

```
arrays = [["X", "X", "Y"], [1, 2, 1]]
index = pd.MultiIndex.from_arrays(arrays)
df = pd.DataFrame({"Score": [80, 90, 70]}, index=index)
print(df.sort_index())
```

Example: Sort by date index

```
dates = pd.to_datetime(["2023-01-03", "2023-01-01", "2023-01-02"])
df = pd.DataFrame({"Value": [300, 100, 200]}, index=dates)
print(df.sort_index())
```

Example: Sort by column index

```
df = pd.DataFrame([[1, 2], [3, 4]], columns=["B", "A"])
print(df.sort_index(axis=1))
```

Example: Sort by custom labels

```
df = pd.DataFrame({"Score": [40, 50, 30]}, index=["Ravi", "Neha", "Amit"])
print(df.sort_index())
```

Example: Sort after changing index

```
df = pd.DataFrame({"Roll": [102, 101, 103], "Marks": [70, 90, 60]})
df = df.set_index("Roll")
print(df.sort_index())
```

Example: Sort hierarchical index (columns)

```
df = pd.DataFrame({"Math", 2022): [70], ("Math", 2021): [80]})
df.columns = pd.MultiIndex.from_tuples(df.columns)
print(df.sort_index(axis=1))
```

.apply()

- .apply() is used to apply a function along rows or columns of a DataFrame or across all elements of a Series.
- More flexible than .map() or .applymap() as it can be used with row-wise or column-wise operations.
- Applicable to both Series & DataFrames.

Syntax

```
Series.apply(func)
DataFrame.apply(func, axis=0)      # axis=0 for columns, axis=1 for rows
```

.apply() — Series Example

```
import pandas as pd
s = pd.Series([2, 4, 6, 8, 10])
```

Example: Square each element

```
print(s.apply(lambda x: x ** 2))
```

Example: Check even or odd

```
print(s.apply(lambda x: 'Even' if x % 2 == 0 else 'Odd'))
```

Example: Convert numbers to strings

```
print(s.apply(str))
```

Example: Multiply all by 1.5

```
print(s.apply(lambda x: x * 1.5))
```

Example: Custom condition-based label

```
def categorize(x):
    return 'High' if x > 5 else 'Low'
print(s.apply(categorize))
```

Sample DataFrame

```
df = pd.DataFrame({  
    'A': [2, 4, 6],  
    'B': [1, 3, 5]  
})
```

Example: Column-wise sum

```
print(df.apply(sum, axis=0))
```

Example: Row-wise max

```
print(df.apply(max, axis=1))
```

Example: Multiply every column by 2

```
print(df.apply(lambda col: col * 2))
```

Example: Add values of A and B row-wise

```
print(df.apply(lambda row: row['A'] + row['B'], axis=1))
```

Example: Conditional transformation: add 10 to numbers < 5

```
print(df.apply(lambda col: col.apply(lambda x: x + 10 if x < 5 else x)))
```

.map()

- .map() is used to apply a function, dictionary, or Series to each element of a Series.
- Cannot be used on DataFrames directly.
- Applicable to Series only.

Syntax

```
Series.map(func_or_dict)
```

.map() — Series Example

```
s = pd.Series([10, 20, 30, 40])
```

Example: Add 5 to each

```
print(s.map(lambda x: x + 5))
```

Example: Multiply by 10

```
print(s.map(lambda x: x * 10))
```

Example: Use dictionary mapping

```
s2 = pd.Series(['A', 'B', 'C', 'A'])  
mapping = {'A': 'Apple', 'B': 'Banana', 'C': 'Cherry'}  
print(s2.map(mapping))
```

Example: Convert to string

```
print(s.map(str))
```

Example: Custom function to double

```
def double(x):  
    return x * 2  
print(s.map(double))
```

Not valid for DataFrames: You cannot use `.map()` directly on a DataFrame. You must apply it to each Series inside the DataFrame using `.apply()`.

.applymap()

- `.applymap()` is used to apply a function element-wise to all cells in a DataFrame.
- Works only on DataFrames, not on Series.
- Applicable to DataFrame only

Syntax

```
DataFrame.applymap(func)
```

.applymap() — DataFrame Only

```
df = pd.DataFrame({  
    'Math': [80, 90, 70],  
    'Science': [85, 95, 75]  
})
```

Example: Add 5 to each cell

```
print(df.applymap(lambda x: x + 5))
```

Example: Convert to percentage string

```
print(df.applymap(lambda x: f"{x}%"))
```

Example: Square each value

```
print(df.applymap(lambda x: x ** 2))
```

Example: Pass/Fail condition

```
print(df.applymap(lambda x: 'Pass' if x >= 75 else 'Fail'))
```

Example: Categorize values

```
print(df.applymap(lambda x: 'High' if x > 80 else 'Low'))
```

Binning and Categorizing in Pandas

Binning refers to converting continuous numerical data into discrete intervals or categories (bins). It helps in:

- Simplifying data
- Visualizing distributions
- Grouping values for analysis

Categorizing means assigning meaningful labels to bins or classifying data based on conditions.

pd.cut()

pd.cut() is a binning or discretization function in Pandas used to convert continuous numerical data into categorical bins (intervals). It is commonly used when:

- You want to group numeric values into ranges
- You want to create labels or classes based on numerical values (e.g., "Low", "Medium", "High")
- You want to simplify numeric data for better visualization or analysis

Real-World Examples

- **Student Marks** → "Fail", "Average", "Excellent"
- **Age** → "Child", "Teen", "Adult", "Senior"
- **Income** → "Low Income", "Middle Class", "High Income"

How pd.cut() works

Internally, pd.cut():

1. Divides your numeric data into bins (ranges of values).
2. Each value in your Series is assigned to a bin based on where it falls.
3. Returns a Categorical Series, where each value is an Interval object or a label, depending on your settings.

Syntax

```
pd.cut(x, bins, right=True, labels=None, retbins=False, precision=3, include_lowest=False, duplicates='raise', ordered=True)
```

Parameter	Description
x	1D array-like numeric data (usually a Series or column)
bins	Number of bins (int) or exact bin edges (list of numbers)
right	If True, bins include the right edge (e.g., (a, b]); else (a, b) is left-inclusive
labels	Labels for the bins (list or False to return bin numbers)
include_lowest	Whether the first bin includes the left edge (useful for edge values)
retbins	If True, returns the bin edges as a tuple (binned, bins)
precision	Precision for bin labels (e.g., 2 decimal places)
duplicates	Controls error on duplicate bin edges. Use 'drop' to silently ignore
ordered	Whether the resulting categorical type should be ordered

How Intervals are Formed

- By default, `pd.cut()` produces right-inclusive intervals: (a, b]

Example

- `pd.cut([5, 10, 15], bins=[0, 10, 20])`

Output: [(0, 10], (0, 10], (10, 20]]

To make intervals 'left-inclusive', use `right=False`:

`pd.cut([5, 10, 15], bins=[0, 10, 20], right=False)`

Output: [(0, 10), [10, 20), [10, 20)]

Example: Basic Equal-Width Binning (4 bins)

```
import pandas as pd
```

```
s = pd.Series([15, 22, 36, 49, 53, 60, 74, 85, 91, 99])
```

```
print(pd.cut(s, bins=4))
```

Example: Custom Bin Edges

```
print(pd.cut(s, bins=[0, 25, 50, 75, 100]))
```

Example: With Labels

```
labels = ['Low', 'Medium', 'High', 'Very High']
```

```
print(pd.cut(s, bins=[0, 25, 50, 75, 100], labels=labels))
```

Example: Include Lowest

```
print(pd.cut(s, bins=[15, 50, 100], include_lowest=True))
```

Example: Binning with `right=False` (exclude right edge)

```
print(pd.cut(s, bins=[0, 25, 50, 75, 100], right=False))
```

Example: Binning into 3 equal bins with labels

```
print(pd.cut(s, bins=3, labels=['Low', 'Medium', 'High']))
```

Example: Add result as new column to DataFrame

```
df = pd.DataFrame({'Score': s})
```

```
df['Grade'] = pd.cut(df['Score'], bins=[0, 30, 70, 100], labels=['Poor', 'Average', 'Excellent'])
```

```
print(df)
```

Example: Show interval ranges

```
cut_result = pd.cut(s, bins=4)
```

```
print(cut_result.categories)
```

Example: Use `value_counts()` on binned data

```
print(pd.cut(s, bins=[0, 50, 75, 100]).value_counts())
```

Example: Use groupby() on binned categories

```
df = pd.DataFrame({'Marks': s})
df['Level'] = pd.cut(df['Marks'], bins=[0, 50, 100], labels=['Below Avg', 'Above Avg'])
print(df.groupby('Level').mean())
```

Example: Binning score into 3 bins using DataFrames

```
data = {
    'Name': ['Amit', 'Raj', 'Suman', 'Kiran', 'Farhan', 'Meena', 'Tanvi', 'Harsh', 'Neha', 'Ravi'],
    'Age': [23, 27, 35, 45, 22, 31, 55, 38, 29, 40],
    'Score': [55, 72, 88, 45, 30, 60, 77, 83, 95, 50]
}
df = pd.DataFrame(data)
df['Score_Level'] = pd.cut(df['Score'], bins=3)
print(df)
```

Example: Label the score bins

```
labels = ['Low', 'Medium', 'High']
df['Score_Level'] = pd.cut(df['Score'], bins=3, labels=labels)
print(df)
```

Example: Binning Age into custom groups

```
df['Age_Group'] = pd.cut(df['Age'], bins=[20, 30, 40, 60], labels=['Youth', 'Adult', 'Senior'])
print(df)
```

Example: Use groupby() on Age_Group

```
print(df.groupby('Age_Group')['Score'].mean())
```

Example: Add multiple binned columns

```
df['Age_Bin'] = pd.cut(df['Age'], bins=4)
df['Score_Bin'] = pd.cut(df['Score'], bins=4)
print(df)
```

Example: Use value_counts() on binned score

```
print(df['Score_Level'].value_counts())
```

Example: Include lowest in Age binning

```
df['Age_Bin_Lowest'] = pd.cut(df['Age'], bins=[22, 30, 40, 60], include_lowest=True)
print(df)
```

Example: Right-edge excluded

```
df['Score_Bin_RightFalse'] = pd.cut(df['Score'], bins=[30, 60, 90, 100], right=False)
print(df)
```

Example: Show interval ranges used

```
cut_result = pd.cut(df['Score'], bins=3)
print(cut_result.cat.categories)
```

Example: Nested Binning (combine Score & Age bins)

```
df['Age_Bin'] = pd.cut(df['Age'], bins=3, labels=['Young', 'Mid-Age', 'Old'])
df['Score_Bin'] = pd.cut(df['Score'], bins=3, labels=['Low', 'Medium', 'High'])
print(df[['Name', 'Age', 'Score', 'Age_Bin', 'Score_Bin']])
```

pd.qcut()

- `pd.qcut()` is a quantile-based discretization function in Pandas.
- It is used to divide a continuous variable into equal-sized quantile bins, meaning each bin will have (approximately) the same number of observations.
- While `pd.cut()` creates equal-width bins, `pd.qcut()` creates equal-frequency bins.

When to use `pd.qcut()`

Use `pd.qcut()` when:

- You want each bin to have an equal number of data points.
- Your data is skewed and you want balanced class sizes.
- You're preparing data for quantile-based analysis or ranking.

Quantile-based analysis is a statistical method that divides data into equal-sized groups (quantiles) based on their rank or distribution, allowing comparisons across percentiles (e.g., top 25%, median, bottom 10%). It helps analyze how values are distributed and identify patterns within specific segments of data.

Syntax

```
pd.qcut(x, q, labels=None, retbins=False, precision=3, duplicates='raise')
```

Parameter	Description
<code>x</code>	The numeric data (array, Series, etc.)
<code>q</code>	Number of quantiles (int), or list of quantile values (e.g., [0, 0.25, 0.5, 0.75, 1])
<code>labels</code>	Optional labels for the bins. If False, returns bin indices
<code>retbins</code>	If True, returns the bin edges as well
<code>precision</code>	Decimal precision for labels
<code>duplicates</code>	If bin edges are not unique, use 'drop' to remove duplicates

pd.cut() vs pd.qcut()

Feature	<code>pd.cut()</code>	<code>pd.qcut()</code>
Bin Type	Equal width (fixed range)	Equal frequency (same number of elements)
Input	Number of bins or fixed edges	Number of quantiles or custom quantile points
Distribution	Ignores distribution	Uses quantiles from distribution
Ideal Use	Uniform distributions	Skewed distributions

Example: Divide into 4 Quantile Bins (quartiles)

```
import pandas as pd
import numpy as np
s = pd.Series([12, 34, 22, 44, 55, 38, 18, 49, 63, 41])
print(pd.qcut(s, q=4))
```

Example: Label the Bins

```
print(pd.qcut(s, q=4, labels=['Q1', 'Q2', 'Q3', 'Q4']))
```

Example: Return Bin Indices

```
print(pd.qcut(s, q=4, labels=False))
```

Example: Return Bin Edges

```
result, bins = pd.qcut(s, q=4, retbins=True)
print(result)
print(bins)
```

Example: Use Custom Quantile Values

```
print(pd.qcut(s, q=[0, 0.3, 0.6, 0.9, 1], labels=["Low", "Medium", "High", "Top"]))
```

Example: Binning with Duplicate Bin Edges

```
s2 = pd.Series([1, 2, 2, 2, 3, 4, 5, 5, 5, 6])
print(pd.qcut(s2, q=4, duplicates='drop'))          # Drops overlapping bins
```

Example: Use value_counts() to inspect group sizes

```
qbins = pd.qcut(s, q=4)
print(qbins.value_counts())
```

Example: Combine with groupby() to find mean of each quantile

```
df = pd.DataFrame({'Score': s})
df['Quantile'] = pd.qcut(df['Score'], q=4)
print(df.groupby('Quantile')['Score'].mean())
```

Example: Different bin sizes with unequal data

```
s3 = pd.Series([1, 10, 15, 22, 29, 36, 45, 60, 73, 88])
print(pd.qcut(s3, q=3, labels=['Low', 'Mid', 'High']))
```

Example: Assign percentiles manually

```
percentiles = pd.qcut(s, q=[0, 0.25, 0.5, 0.75, 1], labels=['25%', '50%', '75%', '100%'])
print(percentiles)
```

Example: Quantile-based Binning on Income using DataFrame

```
data = {
    'Name': ['Amit', 'Raj', 'Suman', 'Kiran', 'Farhan', 'Meena', 'Tanvi', 'Harsh', 'Neha', 'Ravi'],
    'Age': [22, 24, 30, 34, 27, 31, 25, 28, 35, 29],
    'Income': [25000, 30000, 50000, 52000, 27000, 47000, 28000, 36000, 54000, 39000]
}
df = pd.DataFrame(data)
df['Income_Group'] = pd.qcut(df['Income'], q=4)
print(df)
```

Example: Use Labels for Income Brackets

```
labels = ['Low', 'Mid', 'High', 'Very High']
df['Income_Level'] = pd.qcut(df['Income'], q=4, labels=labels)
print(df)
```

Example: Bin by Age into 5 equal groups

```
df['Age_Quantile'] = pd.qcut(df['Age'], q=5)
print(df)
```

Example: Group by Quantile and Calculate Mean Income

```
df['Income_Quantile'] = pd.qcut(df['Income'], q=4)
print(df.groupby('Income_Quantile')['Income'].mean())
```

Example: Use value_counts() on Quantile Bins

```
df['Income_Quantile'] = pd.qcut(df['Income'], q=4)
print(df['Income_Quantile'].value_counts())
```

Example: Retain Bin Ranges with retbins=True

```
qcut_result, bin_edges = pd.qcut(df['Income'], q=4, retbins=True)

print(qcut_result)
print(bin_edges)
```

Example: Bin Index Instead of Labels

```
df['Income_BinID'] = pd.qcut(df['Income'], q=4, labels=False)
print(df)
```

Example: Binning with Duplicate Incomes (use duplicates='drop')

```
df2 = df.copy()
df2['Income'] = [30000, 30000, 30000, 30000, 30000, 30000, 30000, 30000, 30000,
30000]
df2['Income_Group'] = pd.qcut(df2['Income'], q=4, duplicates='drop')

print(df2)
```

Example: Create Multiple Quantile Columns

```
df['Q1'] = pd.qcut(df['Age'], q=4, labels=['A', 'B', 'C', 'D'])
df['Q2'] = pd.qcut(df['Income'], q=4, labels=['Low', 'Medium', 'High', 'Very High'])
print(df[['Name', 'Q1', 'Q2']])
```

Example: Visualize Group Distribution Using groupby()

```
df['Income_Quartile'] = pd.qcut(df['Income'], q=4)
summary = df.groupby('Income_Quartile').agg({'Name': 'count', 'Income': 'mean'})
print(summary)
```

Handle ties or Identical values

When using `pd.qcut()`, ties or identical values can cause duplicate bin edges, which may result in errors like:

ValueError: Bin edges must be unique: array([...])

You can drop duplicate edges by setting the `'duplicates'` kwarg

Why do ties cause errors?

- `pd.qcut()` tries to split data into equal-sized groups based on quantiles.
- If too many identical values exist, it cannot create distinct bins because multiple values fall on the same quantile boundary.
- This results in duplicate bin edges.

Solution

Use the parameter:

`duplicates='drop'`

This tells `qcut()` to drop non-unique bins and continue with fewer bins.

Example: Without `duplicates='drop'` → Error due to ties

```
import pandas as pd
```

```
s = pd.Series([100, 100, 100, 100, 100, 100, 100, 100])
pd.qcut(s, q=4) # This will raise an error
```

Example: Handling ties using `duplicates='drop'`

```
print(pd.qcut(s, q=4, duplicates='drop'))
```

Example: Mixed unique + identical values

```
s = pd.Series([10, 20, 20, 20, 30, 30, 40, 50, 50, 50])
print(pd.qcut(s, q=4, duplicates='drop'))
```

Example: Drop duplicates and assign labels

```
labels = ['Low', 'Medium', 'High']  
print(pd.qcut(s, q=3, labels=labels, duplicates='drop'))
```

Example: Get fewer bins than requested due to ties

```
s = pd.Series([1, 1, 1, 2, 2, 2, 3, 3, 3])  
result = pd.qcut(s, q=4, duplicates='drop')  
print(result.unique())
```

Fewer than 4 bins

Example: Return bin index instead of labels when dropping duplicates

```
print(pd.qcut(s, q=4, labels=False, duplicates='drop'))
```

Example: Check number of observations per bin using value_counts()

```
q = pd.qcut(s, q=3, duplicates='drop')  
print(q.value_counts())
```

Example: Use retbins=True to view adjusted bin edges

```
result, bins = pd.qcut(s, q=4, retbins=True, duplicates='drop')  
print(bins)
```

Example: Apply qcut on DataFrame with duplicate incomes

```
df = pd.DataFrame({'Income': [30000]*10})  
df['Income_Quantile'] = pd.qcut(df['Income'], q=4, duplicates='drop')  
print(df)
```

Example: Grouping and Aggregation after handling ties

```
df = pd.DataFrame({'Score': [50, 50, 50, 50, 70, 80, 80, 90, 100, 100]})  
df['Quantile'] = pd.qcut(df['Score'], q=4, duplicates='drop')  
print(df.groupby('Quantile')['Score'].mean())
```

Projects Ideas where you can apply both `pd.cut()` and `pd.qcut()` effectively

1. Customer Segmentation for Marketing Campaign

Objective: Group customers by income levels, purchase frequency, and spending power to target offers.

Use `pd.cut()` for: Segmenting income into fixed ranges: e.g., Low: 0–30k, Mid: 30k–60k, High: 60k+

Use `pd.qcut()` for: Dividing purchase frequency into equal-sized customer groups (quartiles or deciles)

Sample Data Columns: CustomerID, Age, Income, Purchases, AverageOrderValue, TotalSpending

Output Insight: Create targeted campaigns for top 25% spenders and loyal buyers.

2. Student Performance Analysis

Objective: Analyze student scores to assign grades and percentile ranks.

Use `pd.cut()` for: Assigning grades: e.g., <40 = Fail, 40–60 = Average, >60 = Good

Use `pd.qcut()` for: Ranking students by percentiles: Top 10%, Top 25%, Median scorers

Sample Data Columns: StudentID, Subject, TotalMarks, Percentage, Rank

Output Insight: Find students in the bottom 25% for remedial classes and top 10% for awards.

3. HR Analytics – Salary and Experience Bands

Objective: Analyze employees by experience, salary, and performance.

Use `pd.cut()` for: Defining experience brackets: <2 yrs, 2–5 yrs, >5 yrs

Use `pd.qcut()` for: Breaking salaries into equal-sized bands for fairness review or promotions

Sample Data Columns: EmployeeID, YearsOfExperience, Salary, PerformanceRating

Output Insight: Identify if high performers are being underpaid compared to peers in same quantile.

4. E-Commerce Product Pricing Strategy

Objective: Group products for discount tiers, premium offers, and inventory push.

Use `pd.cut()` for: Categorizing price into manual buckets: Budget, Standard, Premium

Use `pd.qcut()` for: Grouping by sales volume or revenue to target top/mid/low-selling products

Sample Data Columns: ProductID, Price, UnitsSold, Revenue, DiscountOffered

Output Insight: Identify products with low revenue but high volume for upselling campaigns.

5. Website Analytics – Session Time and Bounce Rate

Objective: Understand user behavior based on time spent and bounce rate.

Use `pd.cut()` for: Grouping session time into fixed buckets: <30s, 30s–2min, 2–5min, >5min

Use `pd.qcut()` for: Segmenting bounce rates or page depth into quartiles for trend analysis

Sample Data Columns: SessionID, TimeOnSite, PagesVisited, BounceRate, Conversion

Output Insight: Correlate time-on-site quantiles with conversion to optimize UX.

Grouping and Aggregation in Pandas

Grouping: Grouping means splitting your data into groups based on some criteria, like values in a column (e.g., gender, product category, etc.).

Aggregation: Aggregation refers to computing summary statistics like:

- sum, mean, min, max, count, std, median, etc.

Together, **grouping + aggregation** allows you to:

- Summarize and analyze large datasets
- Understand patterns and trends
- Perform data reduction for visualization or ML

Split-Apply-Combine Strategy

The logic behind grouping and aggregation in Pandas is based on the Split–Apply–Combine strategy:

Step	Description
Split	Split the data into groups based on column(s)
Apply	Apply a function (e.g., mean, sum, count) to each group
Combine	Combine the results into a new DataFrame or Series

This is implemented using the `groupby()` method in Pandas.

.groupby() in Pandas

The `.groupby()` method in Pandas is used to split data into groups based on one or more keys (like column values), then apply aggregation or transformation operations on each group individually.

Syntax

```
df.groupby('column_name')
df.groupby(['col1', 'col2'])['target_col'].agg('sum')
```

Using Series

```
import pandas as pd
s = pd.Series([100, 200, 300, 150, 250], index=['A', 'A', 'B', 'B', 'C'])
```

Example: Group by index

```
print(s.groupby(s.index).sum())
```

Example: Group by index and find mean

```
print(s.groupby(s.index).mean())
```

Example: Group by index and count items

```
print(s.groupby(s.index).count())
```

Example: Group by index and find max

```
print(s.groupby(s.index).max())
```

Example: Apply multiple aggregations

```
print(s.groupby(s.index).agg(['sum', 'mean', 'max']))
```

Example: Group by external Series

```
groups = pd.Series(['X', 'X', 'Y', 'Y', 'Z'], index=s.index)
print(s.groupby(groups).sum())
```

Example: Group by a function on index

```
print(s.groupby(lambda x: x in ['A', 'B']).mean())
```

Example: Group by boolean condition

```
print(s.groupby(s > 200).sum())
```

Example: Group by string length of index

```
print(s.groupby(s.index.str.len()).mean())
```

Example: Group by sorted index

```
print(s.sort_index().groupby(level=0).sum())
```

Using DataFrame

```
data = {
    'Department': ['Sales', 'Sales', 'HR', 'HR', 'IT'],
    'Employee': ['Neha', 'Tushar', 'Garima', 'Shweta', 'Nitin'],
    'Salary': [50000, 60000, 45000, 48000, 70000],
    'Age': [25, 30, 28, 27, 35]
}
df = pd.DataFrame(data)
```

Example: Group by single column and sum

```
print(df.groupby('Department')['Salary'].sum())
```

Example: Group by single column and mean

```
print(df.groupby('Department')['Salary'].mean())
```

Example: Group by multiple columns

```
print(df.groupby(['Department', 'Age'])['Salary'].sum())
```

Example: Apply multiple aggregations

```
print(df.groupby('Department')['Salary'].agg(['sum', 'mean', 'max']))
```

Example: Group entire DataFrame

```
print(df.groupby('Department').agg({'Salary': 'mean', 'Age': 'max'}))
```

Example: Use size() for group sizes

```
print(df.groupby('Department').size())
```

Example: Use count() to count non-null rows

```
print(df.groupby('Department').count())
```

Example: Apply a custom function

```
print(df.groupby('Department')['Salary'].apply(lambda x: x.max() - x.min()))
```

Example: Filter groups with more than 1 employee

```
print(df.groupby('Department').filter(lambda x: len(x) > 1))
```

Example: Group and transform (e.g., normalize within group)

```
df['Norm_Salary'] = df.groupby('Department')['Salary'].transform(lambda x: x / x.mean())  
print(df)
```

Aggregation Functions in Pandas

- Aggregation functions in Pandas summarize or reduce data into a single or a few values per group or entire column.
- They are often used with `.groupby()` but can also be used directly on Series or DataFrames.

List of Aggregation Functions

- `sum()` : Total sum of values
- `mean()` : Average of values
- `min()` : Minimum value
- `max()` : Maximum value
- `count()` : Count of non-null values
- `nunique()` : Number of unique values (non-null)

Using Series

```
import pandas as pd  
s = pd.Series([10, 20, 30, 40, 50, 30, 20])
```

`sum()` – Total of all values in Series

Example: Sum of entire Series

```
print("Total Sum:", s.sum())
```


Example: Sum of first 3 values

```
print("Sum of first 3:", s[:3].sum())
```

Example: Sum of values > 25

```
print("Sum > 25:", s[s > 25].sum())
```

Example: Sum of every second value

```
print("Sum at even index:", s[::2].sum())
```

Example: Sum of squared values

```
print("Sum of squares:", (s**2).sum())
```

Example: Sum after subtracting 10

```
print("Sum (s - 10):", (s - 10).sum())
```

Example: Sum of absolute values

```
print("Absolute Sum:", s.abs().sum())
```

Example: Cumulative sum last value

```
print("Last of cumsum:", s.cumsum().iloc[-1])
```

Example: Sum of values divisible by 10

```
print("Sum % 10 == 0:", s[s % 10 == 0].sum())
```

Example: Group-wise sum

```
print(s.groupby(s).sum())
```

mean() – Average value in Series

Example: Mean of entire Series

```
print("Mean:", s.mean())
```

Example: Mean of first 4 items

```
print("Mean first 4:", s[:4].mean())
```

Example: Mean of values < 40

```
print("Mean < 40:", s[s < 40].mean())
```

Example: Mean of even index items

```
print("Mean @ even index:", s[::2].mean())
```

Example: Mean after adding 5

```
print("Mean + 5:", (s + 5).mean())
```

Example: Mean of squared values

```
print("Mean squares:", (s**2).mean())
```

Example: Mean of last 3 values

```
print("Mean of tail:", s.tail(3).mean())
```

Example: Mean of unique values

```
print("Mean unique:", pd.Series(s.unique()).mean())
```

Example: Grouped mean

```
print(s.groupby(s).mean())
```

Example: Mean using describe

```
print("Mean (describe):", s.describe()['mean'])
```

min() – Minimum value in Series

Example: Overall minimum

```
print("Min:", s.min())
```

Example: Min of first 4 items

```
print("Min of first 4:", s[:4].min())
```

Example: Min of items > 25

```
print("Min > 25:", s[s > 25].min())
```

Example: Min at alternate indices

```
print("Min (alt):", s[::2].min())
```

Example: Min of tail

```
print("Tail min:", s.tail().min())
```

Example: Min after adding 5

```
print("Min (s+5):", (s+5).min())
```

Example: Min of squared values

```
print("Min square:", (s**2).min())
```

Example: Min of filtered even values

```
print("Min (even):", s[s % 2 == 0].min())
```

Example: Min of reversed Series

```
print("Min reversed:", s[::-1].min())
```

Example: Group-wise min

```
print(s.groupby(s).min())
```

max() – Maximum value in Series

Example: Overall max

```
print("Max:", s.max())
```

Example: Max of first 5

```
print("Max first 5:", s[:5].max())
```

Example: Max of items < 40

```
print("Max < 40:", s[s < 40].max())
```

Example: Max @ even indices

```
print("Max even index:", s[::2].max())
```

Example: Max of tail

```
print("Max tail:", s.tail().max())
```

Example: Max (s + 10)

```
print("Max +10:", (s+10).max())
```

Example: Max squared

```
print("Max square:", (s**2).max())
```

Example: Max of unique values

```
print("Max unique:", pd.Series(s.unique()).max())
```

Example: Max using describe

```
print("Max (describe):", s.describe()['max'])
```

Example: Group-wise max

```
print(s.groupby(s).max())
```

count() – Number of non-null values

Example: Count all values

```
print("Count:", s.count())
```

Example: Count in first 5

```
print("Count first 5:", s[:5].count())
```

Example: Count where $s > 25$

```
print("Count > 25:", s[s > 25].count())
```

Example: Count with NaNs

```
s2 = pd.Series([10, 20, None, 30])  
print("Count (NaN):", s2.count())
```

Example: Count filtered

```
print("Count even:", s[s % 2 == 0].count())
```

Example: Count of unique values

```
print("Unique values count:", pd.Series(s.unique()).count())
```

Example: Count of $s == 30$

```
print("Count == 30:", (s == 30).sum())
```

Example: Count from boolean mask

```
print("Count from mask:", (s > 10).astype(int).count())
```

Example: Group-wise count

```
print(s.groupby(s).count())
```

Example: Count after dropna

```
print("Count dropna:", s.dropna().count())
```

nunique() – Count of distinct values

Example: Overall unique count

```
print("Unique count:", s.nunique())
```

Example: Unique in first 4

```
print("First 4 unique:", s[:4].nunique())
```

Example: Unique of filtered values

```
print("Unique > 25:", s[s > 25].nunique())
```

Example: Unique of odd values

```
print("Unique odd:", s[s % 2 != 0].nunique())
```

Example: Unique values after square

```
print("Unique squares:", (s**2).nunique())
```

Example: Unique from reversed

```
print("Reversed unique:", s[::-1].nunique())
```

Example: Unique from (s + 10)

```
print("s + 10 unique:", (s + 10).nunique())
```

Example: Unique from tail

```
print("Tail unique:", s.tail().nunique())
```

Example: Count of unique values via set

```
print("Set len:", len(set(s)))
```

Example: Group-wise nunique

```
print(s.groupby(s).nunique())
```

Using DataFrame

```
import pandas as pd
data = {
    'Name': ['Amit', 'Suman', 'Amit', 'Kiran', 'Suman', 'Raj', 'Raj', 'Suman', 'Kiran', 'Raj'],
    'Age': [28, 24, 28, 30, 26, 32, 35, 29, 30, 33],
    'Salary': [30000, 40000, 30000, 35000, 31000, 30000, 30000, 40000, 35000, 30000]
}
df = pd.DataFrame(data)
```

sum() – Total of all values in Series

Example: Sum of all numeric columns

```
print(df.sum(numeric_only=True))
```

Example: Sum of Salary column

```
print(df['Salary'].sum())
```

Example: Sum of Age column

```
print(df['Age'].sum())
```

Example: Sum by Name (Salary)

```
print(df.groupby('Name')['Salary'].sum())
```

Example: Sum by Age (Salary)

```
print(df.groupby('Age')['Salary'].sum())
```

Example: Sum Salary where Age > 28

```
print(df[df['Age'] > 28]['Salary'].sum())
```

Example: Sum of Salary * 2

```
print( (df['Salary'] * 2).sum())
```

Example: Sum by (Name, Age)

```
print(df.groupby(['Name', 'Age'])['Salary'].sum())
```

Example: Sum row-wise (first 3 rows)

```
print(df.iloc[:3].sum(numeric_only=True, axis=1))
```

Example: Sum using agg

```
print(df.groupby('Name').agg({'Salary': 'sum'}))
```

mean() – Average value in Series

Example: Mean of all numeric columns

```
print(df.mean(numeric_only=True))
```

Example: Mean of Salary column

```
print(df['Salary'].mean())
```

Example: Mean by Name (Salary)

```
print(df.groupby('Name')['Salary'].mean())
```

Example: Mean by Age (Salary)

```
print(df.groupby('Age')['Salary'].mean())
```

Example: Mean Salary where Age < 30

```
print(df[df['Age'] < 30]['Salary'].mean())
```

Example: Mean by (Name, Age)

```
print(df.groupby(['Name', 'Age'])['Salary'].mean())
```

Example: Mean row-wise (first 3 rows)

```
print(df.iloc[:3].mean(numeric_only=True, axis=1))
```

Example: Mean using agg

```
print(df.groupby('Name').agg({'Salary': 'mean'}))
```

Example: Mean of Age column

```
print(df['Age'].mean())
```

Example: Mean of Salary + 5000

```
print( (df['Salary'] + 5000).mean())
```

min() – Minimum value in Series

Example: Min of all numeric columns

```
print(df.min(numeric_only=True))
```

Example: Min of Salary column

```
print(df['Salary'].min())
```

Example: Min by Name (Salary)

```
print(df.groupby('Name')['Salary'].min())
```

Example: Min by Age (Salary)

```
print(df.groupby('Age')['Salary'].min())
```

Example: Min of Age column

```
print(df['Age'].min())
```

Example: Min row-wise (first 3 rows)

```
print(df.iloc[:3].min(numeric_only=True, axis=1))
```

Example: Min using agg

```
print(df.groupby('Name').agg({'Salary': 'min'}))
```

Example: Min Salary where Age > 28

```
print(df[df['Age'] > 28]['Salary'].min())
```

Example: Min of Salary * 2

```
print((df['Salary'] * 2).min())
```

Example: Min by (Name, Age)

```
print(df.groupby(['Name', 'Age'])['Salary'].min())
```

max() – Maximum value in Series

Example: Max of all numeric columns

```
print(df.max(numeric_only=True))
```

Example: Max of Salary column

```
print(df['Salary'].max())
```

Example: Max by Name (Salary)

```
print(df.groupby('Name')['Salary'].max())
```

Example: Max by Age (Salary)

```
print(df.groupby('Age')['Salary'].max())
```

Example: Max of Age column

```
print(df['Age'].max())
```

Example: Max row-wise (first 3 rows)

```
print(df.iloc[:3].max(numeric_only=True, axis=1))
```

Example: Max using agg

```
print(df.groupby('Name').agg({'Salary': 'max'}))
```

Example: Max Salary where Age < 30

```
print(df[df['Age'] < 30]['Salary'].max())
```

Example: Max of Salary * 3

```
print((df['Salary'] * 3).max())
```

Example: Max by (Name, Age)

```
print(df.groupby(['Name', 'Age'])['Salary'].max())
```

count() – Number of non-null values

Example: Count of all values in each column

```
print(df.count())
```

Example: Count of non-null values in Salary

```
print(df['Salary'].count())
```

Example: Count by Name (Salary entries)

```
print(df.groupby('Name')['Salary'].count())
```

Example: Count by Age (Salary entries)

```
print(df.groupby('Age')['Salary'].count())
```

Example: Count using agg (Salary)

```
print(df.groupby('Name').agg({'Salary': 'count'}))
```

Example: Count where Salary > 40000

```
print((df['Salary'] > 40000).sum())
```


Example: Count of rows where Age > 30

```
print((df['Age'] > 30).sum())
```

Example: Count of total rows in DataFrame

```
print(len(df))
```

Example: Count of all values by row (first 3 rows)

```
print(df.iloc[:3].count(axis=1))
```

Example: Count of Salary where Name = 'Raj'

```
print(df[df['Name'] == 'Raj']['Salary'].count())
```

nunique() – Count of distinct values

Example: Number of unique values in each column

```
print(df.nunique())
```

Example: nunique() on Name column

```
print(df['Name'].nunique())
```

Example: nunique() on Salary column

```
print(df['Salary'].nunique())
```

Example: nunique() by column using apply

```
print(df.apply(pd.Series.nunique))
```

Example: nunique() by Name (on Salary)

```
print(df.groupby('Name')['Salary'].nunique())
```

Example: nunique() by Age (on Salary)

```
print(df.groupby('Age')['Salary'].nunique())
```

Example: nunique() using agg()

```
print(df.groupby('Name').agg({'Salary': 'nunique'}))
```

Example: nunique() on all selected numeric columns

```
print(df[['Age', 'Salary']].nunique())
```

Example: nunique() with multiple group keys

```
print(df.groupby(['Name', 'Age'])['Salary'].nunique())
```

Example: nunique() on filtered DataFrame

```
print(df[df['Salary'] > 40000]['Name'].nunique())
```